REȚELE DE CALCULATOARE · SEMESTRUL 1 · SĂPTĂMÂNA 3

Nivelul aplicație în scheme

Socket-uri, Web-ul și HTTP/1.1 — curs, seminar și laboratoare

Conf.univ.dr. Popescu-Bodorin Nicolaie, 2026
+40722882471, bodorin(on)live.com, https://lmrec.org/bodorin/

Convenție de terminologie

La prima apariție în fiecare parte a documentului, termenii de specialitate traduși în română sunt urmați, în paranteză, de termenul consacrat în limba engleză sau de forma „romgleză” folosită curent de practicieni — de exemplu, **comutator de pachete** (*packet switch*), **memorie tampon** (*buffer*, în romgleză „buffer”). Comenzile, numele câmpurilor din Wireshark și rezultatele afișate de programe sunt lăsate în engleză, așa cum apar pe ecran.

termen românesc	**termen englezesc**	**formă romgleză**
comutator de pachete	(packet switch)	„buffer”, „socket”

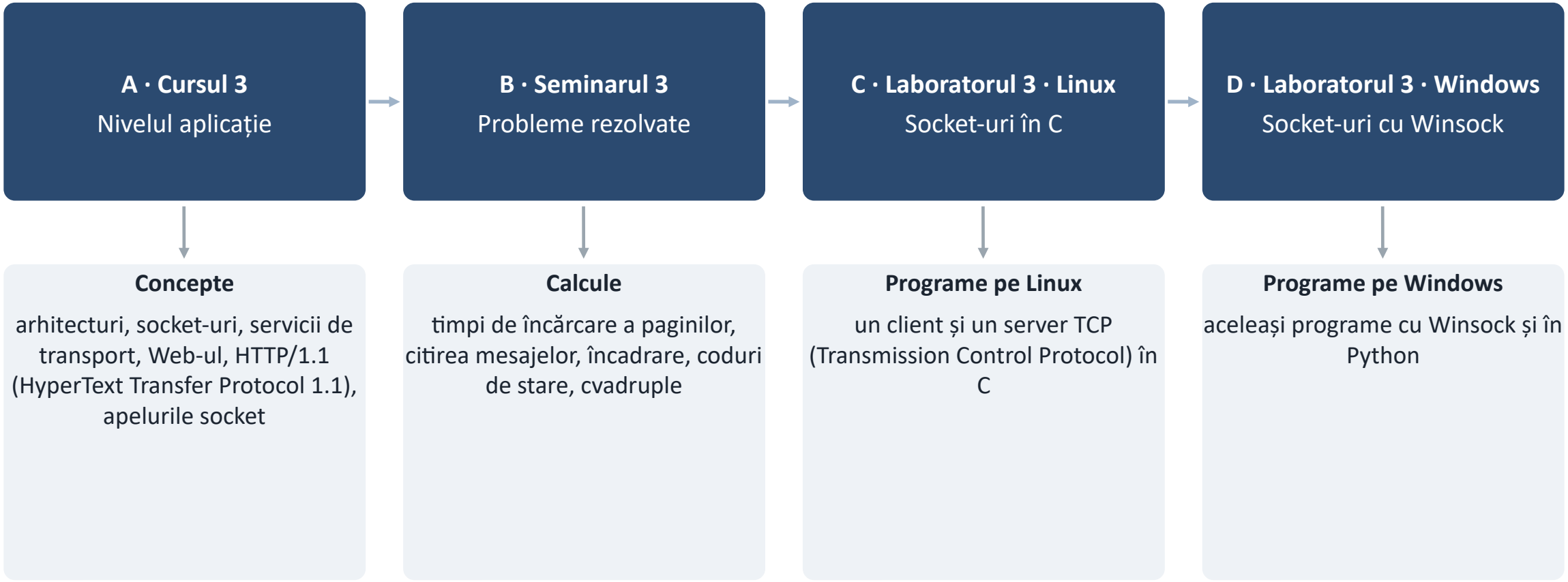
rămân în engleză, așa cum apar pe ecran:

`connect: Connection refused` · `recv returned 65536 bytes` · `Get-NetTCPConnection` · `HTTP/1.1 200 OK`

Ideea-cheie Termenul românesc pentru înțelegere, cel englezesc pentru documentație și pentru motoarele de căutare.

HTTP (HyperText Transfer Protocol)

Cum este organizată săptămâna



Ideea-cheie Citiți prezentarea de la stânga la dreapta: fiecare parte folosește ce a construit cea dinainte.

Partea A · Cursul 3: nivelul aplicație

A · Cursul 3
Nivelul aplicație

→

B · Seminarul 3
Probleme rezolvate

→

C · Laboratorul 3 · Linux
Socket-uri în C

→

D · Laboratorul 3 · Windows
Socket-uri cu Winsock

În această parte

1 · Unde se află nivelul aplicație

2 · Arhitecturi de aplicații

3 · Procese, adrese, socket-uri

4 · Ce le trebuie aplicațiilor de la transport

5 · Ce definește un protocol

6 · Web-ul: obiecte și URL-uri

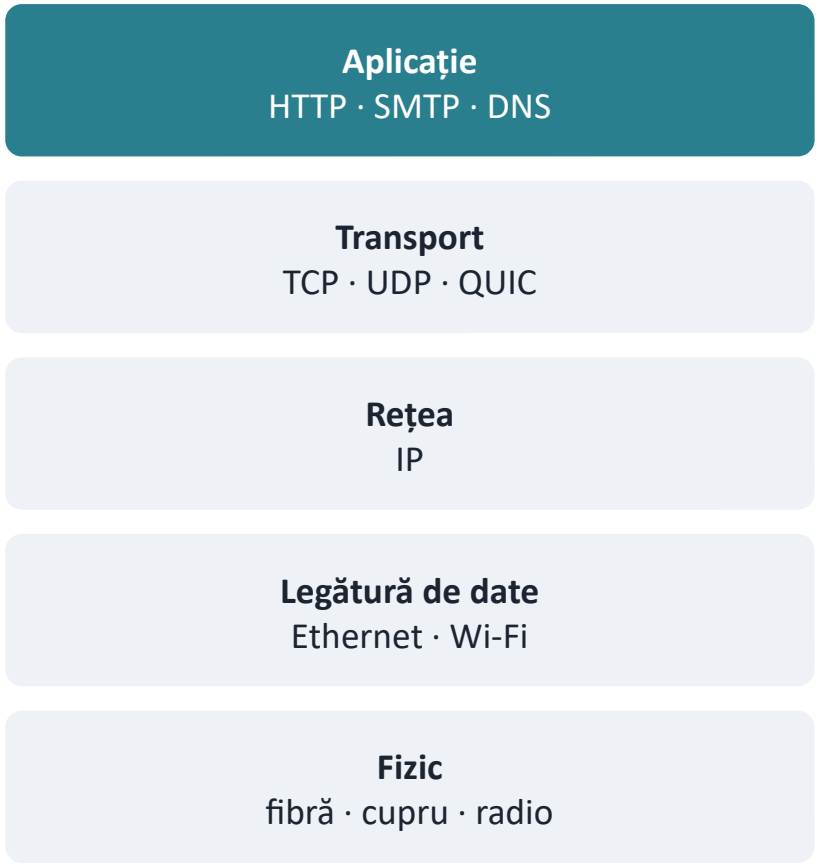
7 · Conexiuni HTTP și timpi dus-întors

8 · Mesajele HTTP/1.1

9 · Apelurile socket

HTTP (HyperText Transfer Protocol) · URL (Uniform Resource Locator)

1 · De la baza stivei spre vârf



1. **Nivelul aplicație** (*application layer*): protocoalele folosite direct de programe — HTTP (HyperText Transfer Protocol), SMTP (Simple Mail Transfer Protocol), DNS (Domain Name System).
2. Sub el: TCP (Transmission Control Protocol), UDP (User Datagram Protocol) și QUIC (protocol de transport peste UDP; numele inițial: Quick UDP Internet Connections) îi transportă mesajele.

săpt. 3 → 14: de sus în jos

3. Rețea: IP (Internet Protocol). Legătură de date și fizic: cadre și semnale.
4. Săptămâna aceasta: **cursul** explică, **laboratorul** programează.

Ideea-cheie Studiem întâi nivelul pentru care veți programa tot restul carierei.

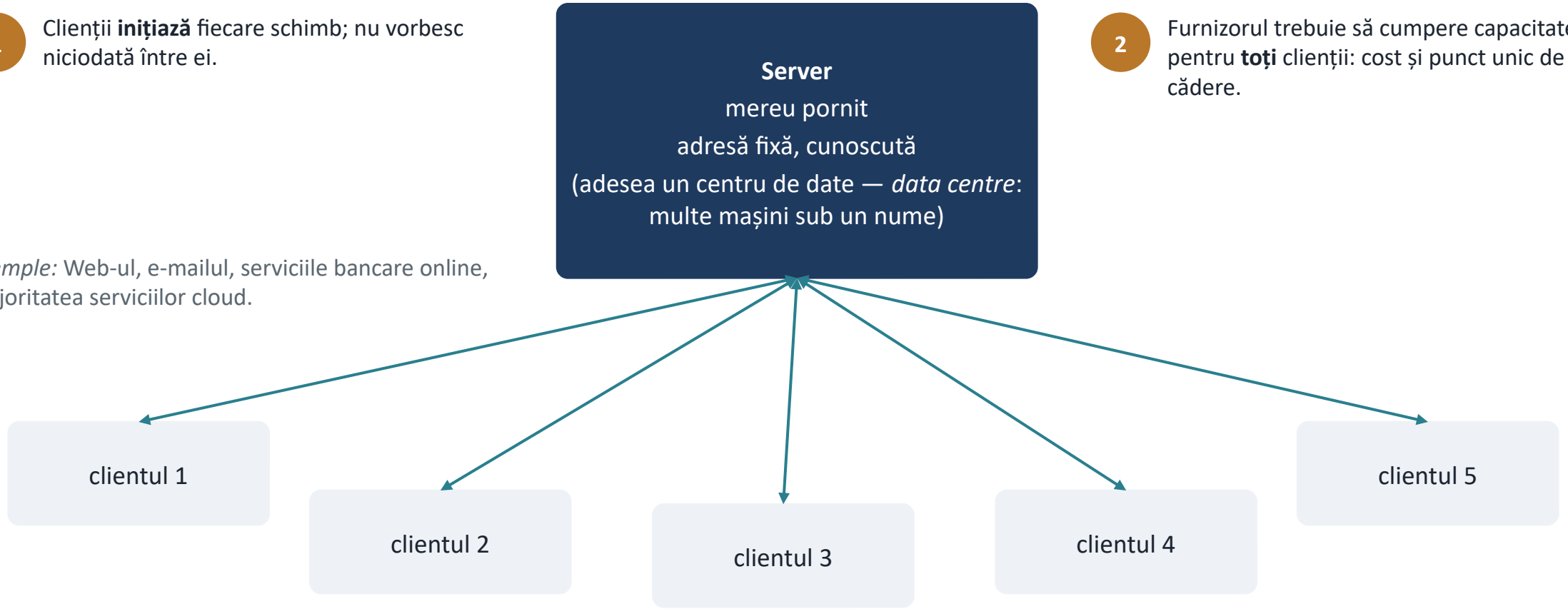
QUIC (protocol de transport peste UDP, User Datagram Protocol; numele inițial: Quick UDP Internet Connections)

2 · Arhitectura client–server

1 Clienții **inițiază** fiecare schimb; nu vorbesc niciodată între ei.

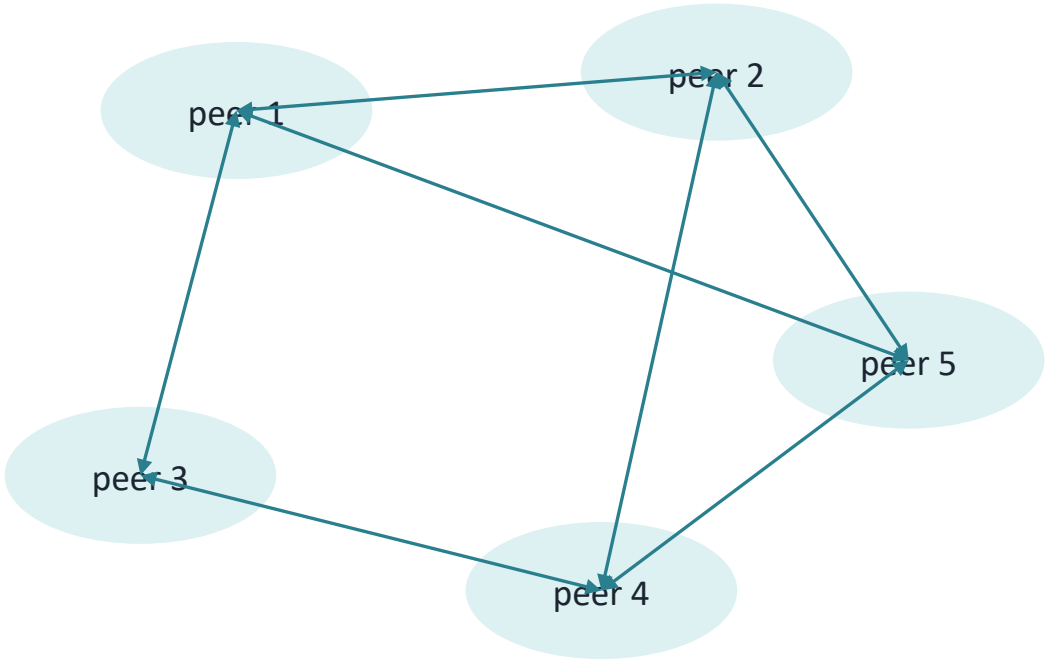
2 Furnizorul trebuie să cumpere capacitate pentru **toți** clienții: cost și punct unic de cădere.

Exemple: Web-ul, e-mailul, serviciile bancare online, majoritatea serviciilor cloud.



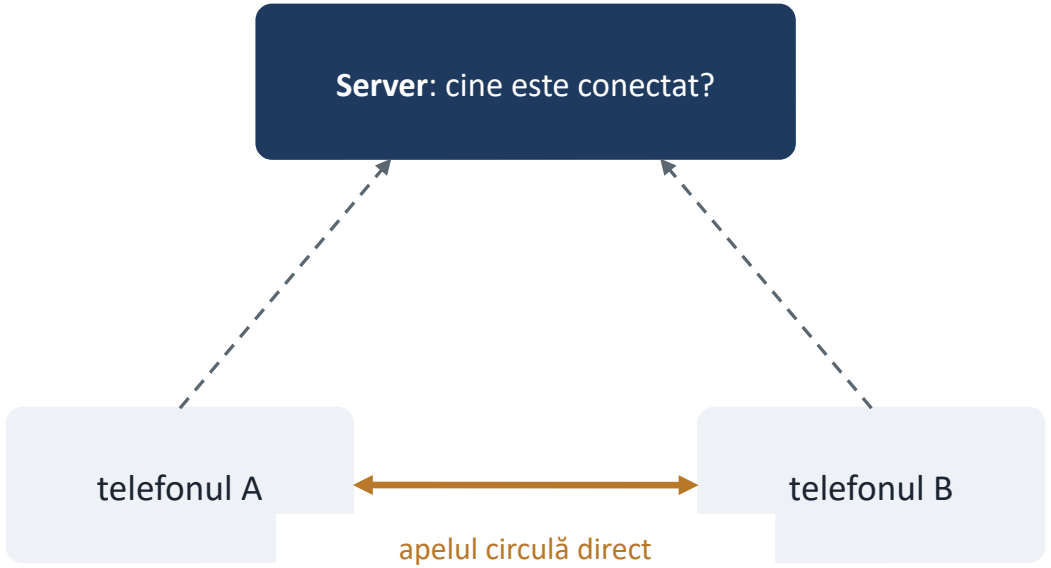
Ideea-cheie Client–server: o parte mereu pornită deservește mulți clienți — ușor de controlat, scump de scalat.

2 · Arhitectura de la egal la egal și sistemele hibride



P2P (peer-to-peer, de la egal la egal): fiecare entitate pereche (*peer*) este client *și* server.

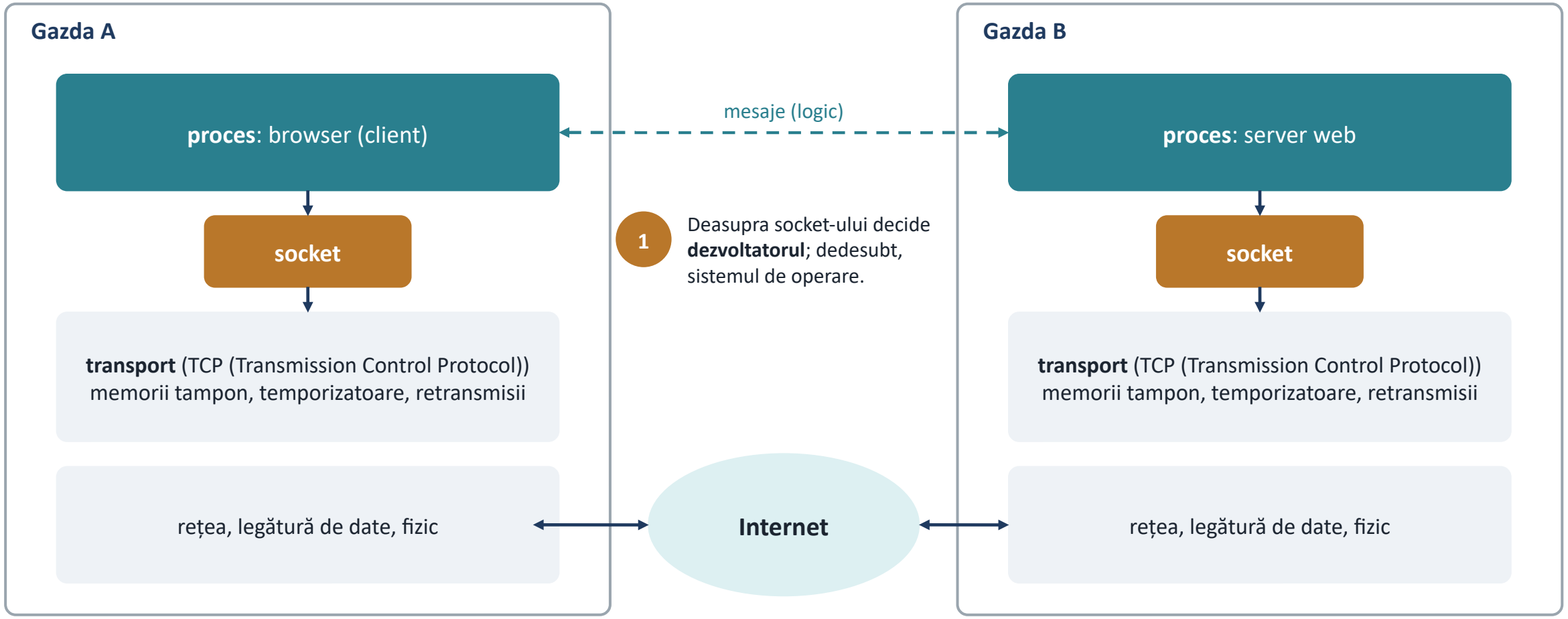
Auto-scalare (*self-scaling*): fiecare entitate nouă aduce cerere *și* capacitate.



1 **Hibrid:** un server îl găsește pe celălalt utilizator (linia punctată); vocea sau imaginea circulă apoi direct între entități.

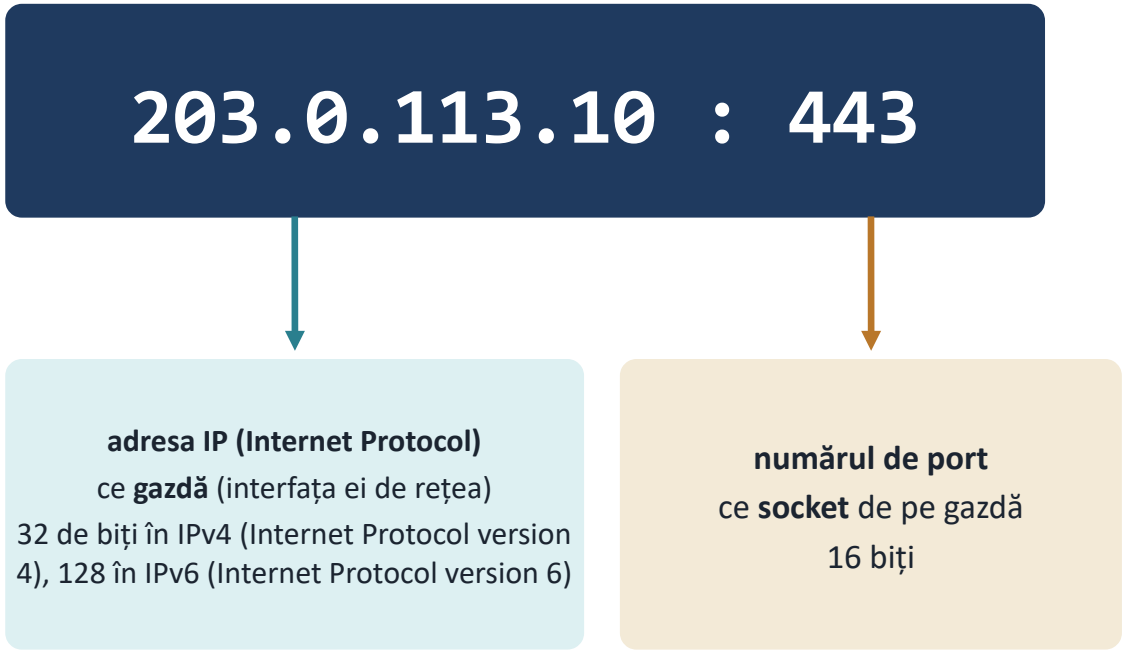
Ideea-cheie Entitățile se scalează singure, dar vin și pleacă, stau în spatele ruterelor și sunt greu de securizat — de aceea sistemele reale le combină.

3 · Procesele comunică prin socket-uri



Ideea-cheie Socket-ul este ușa: procesul împinge un mesaj afară și se bazează pe transport să-l ducă la cealaltă ușă.

3 · Adresarea unui proces: adresă IP + port



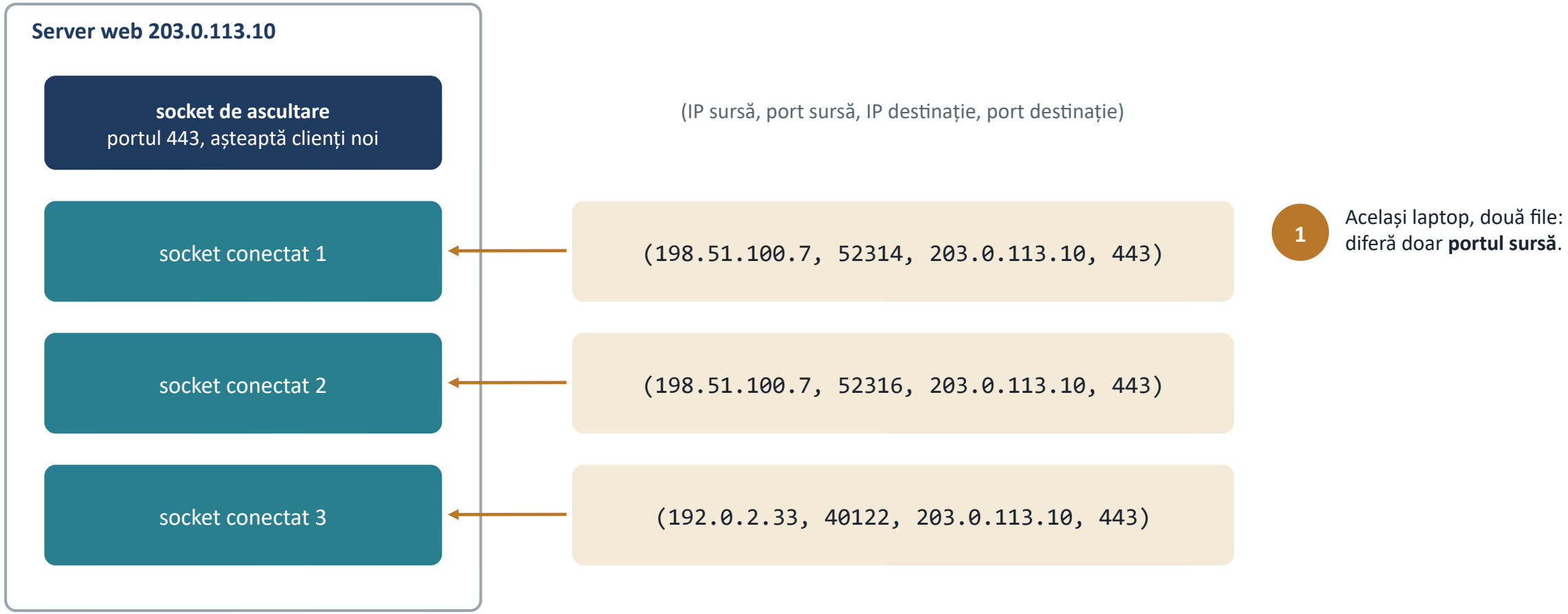
Porturi bine-cunoscute (*well-known ports*, servere)

80	HTTP (HyperText Transfer Protocol)
443	HTTPS (HyperText Transfer Protocol Secure)
53	DNS (Domain Name System)
25	SMTP (Simple Mail Transfer Protocol)
22	SSH (Secure Shell)

1 Socket-ul unui client primește de la sistemul de operare un **port efemer** (*ephemeral port*) temporar (de ex. 51234).

Ideea-cheie Adresa IP = ce mașină; portul = ce program de pe ea. Serverele folosesc porturi bine-cunoscute, clienții porturi temporare.

3 · Cvadruplul deosebește conexiunile



Ideea-cheie O conexiune TCP (Transmission Control Protocol) este numită de cvadruplul ei (*4-tuple*); nu există două conexiuni cu toate cele patru numere identice.

IP (Internet Protocol)

4 · De ce au nevoie aplicațiile de la transport

1 **Livrare fiabilă?**

Trebuie să ajungă fiecare octet, intact și în ordine?

transfer de fișiere: da · apel video: o pierdere mică e acceptabilă

2 **Debit (throughput)?**

O rată minimă sau cât este disponibil (elastic)?

flux video: 5 Mbit/s · e-mail: elastic

3 **Temporizare (timing)?**

Trebuie datele să ajungă într-un termen?

jocuri, voce: zeci de ms · descărcare: nu

4 **Securitate?**

Confidențialitate, integritate, autentificare?

servicii bancare: esențială

Ideea-cheie Răspunsurile la aceste patru întrebări aleg protocolul de transport — nu obișnuința.

4 · Cerințele aplicațiilor obișnuite

Aplicație	Pierdere de date	Debit	Sensibilă la timp
Transfer de fișiere	fără pierderi	elastic	nu
E-mail	fără pierderi	elastic	nu
Pagini web	fără pierderi	elastic	secunde
Apeluri vocale și video	tolerant	kbit/s – Mbit/s	≈ 100 ms
Video înregistrat, în flux	tolerant	Mbit/s	câteva s
Jocuri interactive	tolerant	de la kbit/s	zeci de ms
Mesagerie text	fără pierderi	elastic	și da, și nu

Ideea-cheie Fișierele și e-mailul cer fiecare octet, dar nu viteză; apelurile și jocurile cer viteză, dar tolerează pierderi.

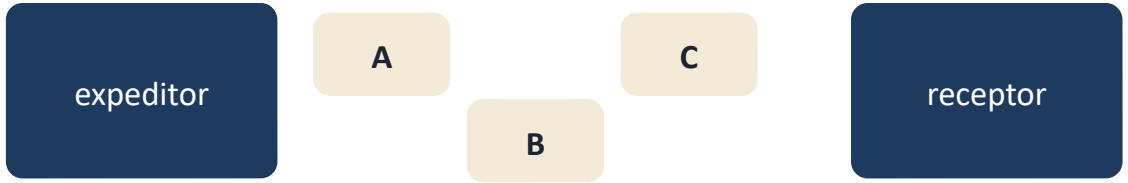
4 · Ce oferă Internetul: TCP și UDP

TCP (Transmission Control Protocol)



- mai întâi o **conexiune** (stabilire SYN (synchronize, sincronizare) / ACK (acknowledgement, confirmare))
- **fiecare octet** ajunge, **în ordine**
- controlul fluxului și al congestiei
- **fără granițe între mesaje** — un singur flux

UDP (User Datagram Protocol)



B se poate pierde; C poate sosi primul

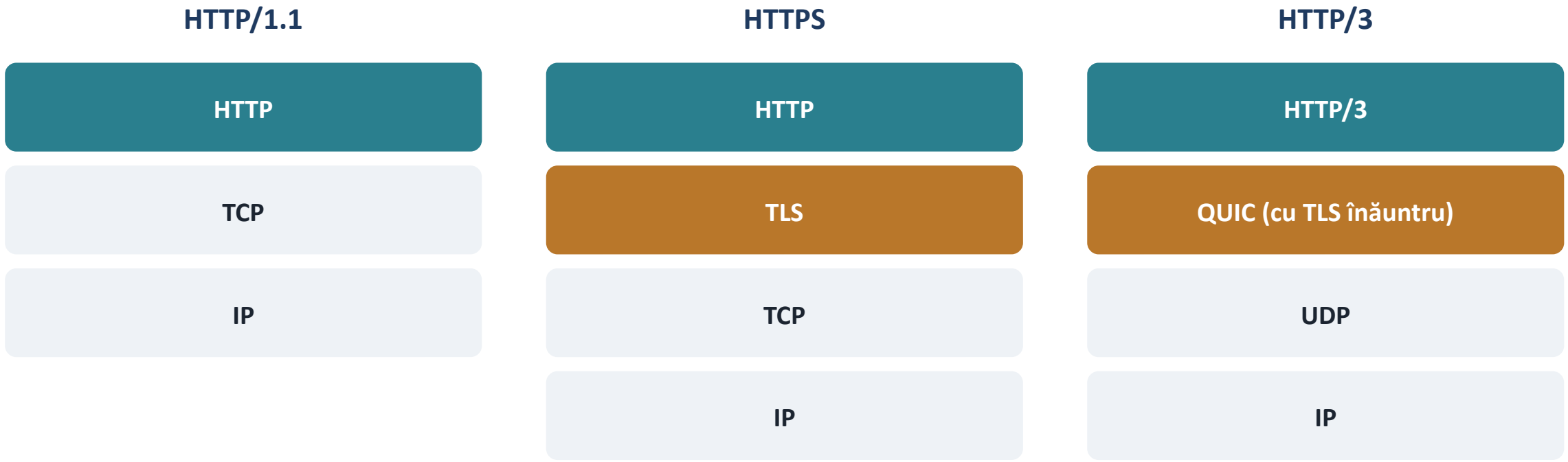
- **fără** conexiune: doar trimite
- datagramele pot fi **pierdute, duplicate, reordonate**
- fără controlul fluxului sau al congestiei
- **granițe păstrate**: un send = o datagramă

Niciunul nu dă garanții de timp sau de debit și niciunul nu criptează. TCP (Transmission Control Protocol): HTTP/1.1 și HTTP/2, SMTP, SSH. UDP (User Datagram Protocol): DNS, voce, video, jocuri, QUIC.

Ideea-cheie TCP = conductă fiabilă și ordonată de octeți; UDP = plicuri independente, fără promisiuni.

HTTP (HyperText Transfer Protocol) · QUIC (protocol de transport peste UDP, User Datagram Protocol; numele inițial: Quick UDP Internet Connections) · SMTP (Simple Mail Transfer Protocol) · DNS (Domain Name System) · SSH (Secure Shell)

4 · Unde se află TLS și QUIC



TLS (Transport Layer Security): o bibliotecă pe care aplicația o apelează în locul socket-ului simplu — criptare și autentificare peste TCP (Transmission Control Protocol). HTTPS (HyperText Transfer Protocol Secure) = HTTP peste TLS.

QUIC (protocol de transport peste UDP; numele inițial: Quick UDP Internet Connections): sub HTTP/3; HTTP (HyperText Transfer Protocol); UDP (User Datagram Protocol); IP (Internet Protocol).

Ideea-cheie Criptarea este un nivel pe care îl adaugi (TLS) — sau pe care îl construiești într-un transport nou (QUIC).

QUIC (protocol de transport peste UDP, User Datagram Protocol; numele inițial: Quick UDP Internet Connections)

4 · De ce ar alege cineva UDP?



TCP (Transmission Control Protocol) mai costă și un timp dus-întors pentru stabilirea conexiunii și poate încetini un expeditor. Peste UDP (User Datagram Protocol) o aplicație adaugă exact fiabilitatea de care are nevoie — sau construiește un transport nou, cum face QUIC.

Ideea-cheie Pentru un joc de curse, o poziție întârziată este mai rea decât una pierdută — de aceea folosește UDP.

QUIC (protocol de transport peste UDP, User Datagram Protocol; numele inițial: Quick UDP Internet Connections)

5 · Ce definește un protocol de nivel aplicație

```
GET /index.html HTTP/1.1
Host: www.example.org

HTTP/1.1 200 OK
Content-Length: 5120
…
```

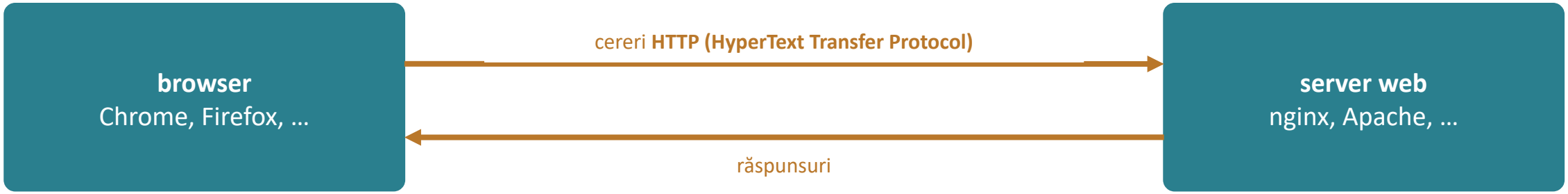
1. **tipuri de mesaje** — cerere · răspuns
2. **sintaxă** — câmpurile și cum sunt delimitate
3. **semantică** — ce înseamnă fiecare câmp (200 = succes)
4. **reguli** — clientul vorbește primul; un răspuns pentru fiecare cerere

Protocoale **deschise** (*open*): în RFC (Request for Comments)-uri publice — HTTP, SMTP, DNS.
Proprietare (*proprietary*): multe aplicații de mesagerie și apeluri.

Ideea-cheie Tipuri, sintaxă, semantică, reguli: o listă de verificare pentru citirea — sau proiectarea — oricărui protocol.

HTTP (HyperText Transfer Protocol) · SMTP (Simple Mail Transfer Protocol) · DNS (Domain Name System)

5 · O aplicație înseamnă mai mult decât protocolul ei



Aplicația Web mai are nevoie de:

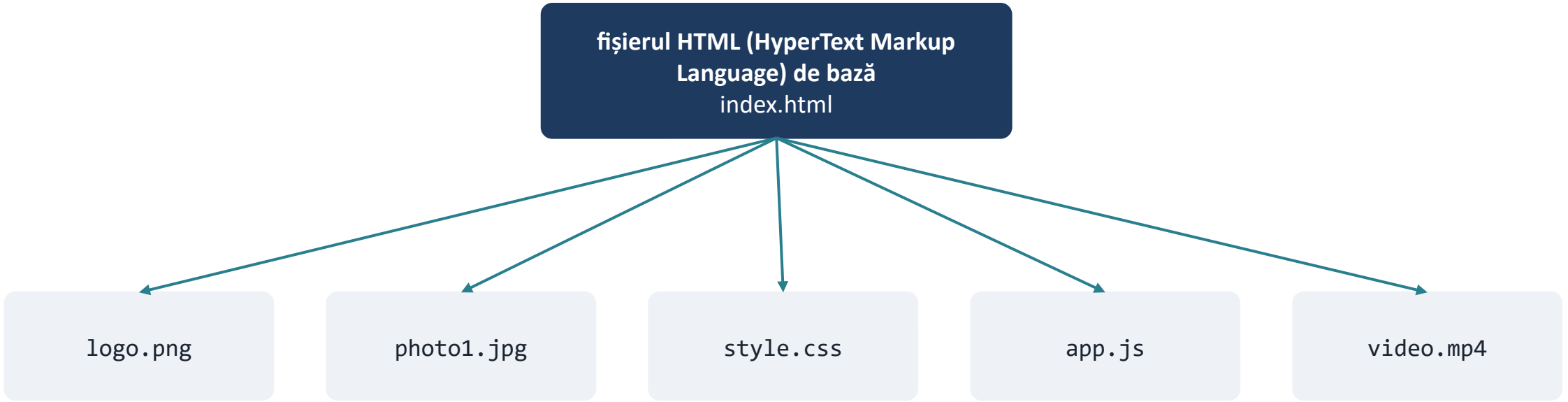
documente
fișiere HTML (HyperText Markup Language)

standarde
HTML, CSS (Cascading Style Sheets), HTTP

adrese
URL (Uniform Resource Locator)-uri

Ideea-cheie Web-ul = browsere + servere + documente + standarde; HTTP este protocolul care le leagă.

6 · O pagină web este un arbore de obiecte



1 Fiecare **obiect** (*object*) este un fișier cu propria adresă și propria cerere.

2 O pagină de știri face astăzi referire la **mult peste o sută** de obiecte.

Ideea-cheie Încărcarea unei pagini înseamnă un fișier de bază și apoi multe obiecte — așa că timpii dus-întors se adună.

6 · Anatomia unui URL

URL (Uniform Resource Locator)



schemă (scheme)
ce protocol: http, https

gazdă (host)
tradusă de DNS (Domain Name System) într-o adresă IP

port
opțional: implicit 80 pentru http, 443 pentru https

cale (path)
ce obiect de pe server

interogare (query)
parametri pentru server

fragment
folosit doar de browser — nu se trimite

Ideea-cheie Schema + gazda + portul găsesc serverul; calea + interogarea numesc obiectul; fragmentul rămâne în browser.

IP (Internet Protocol)

6 · HTTP: cerere și răspuns



1

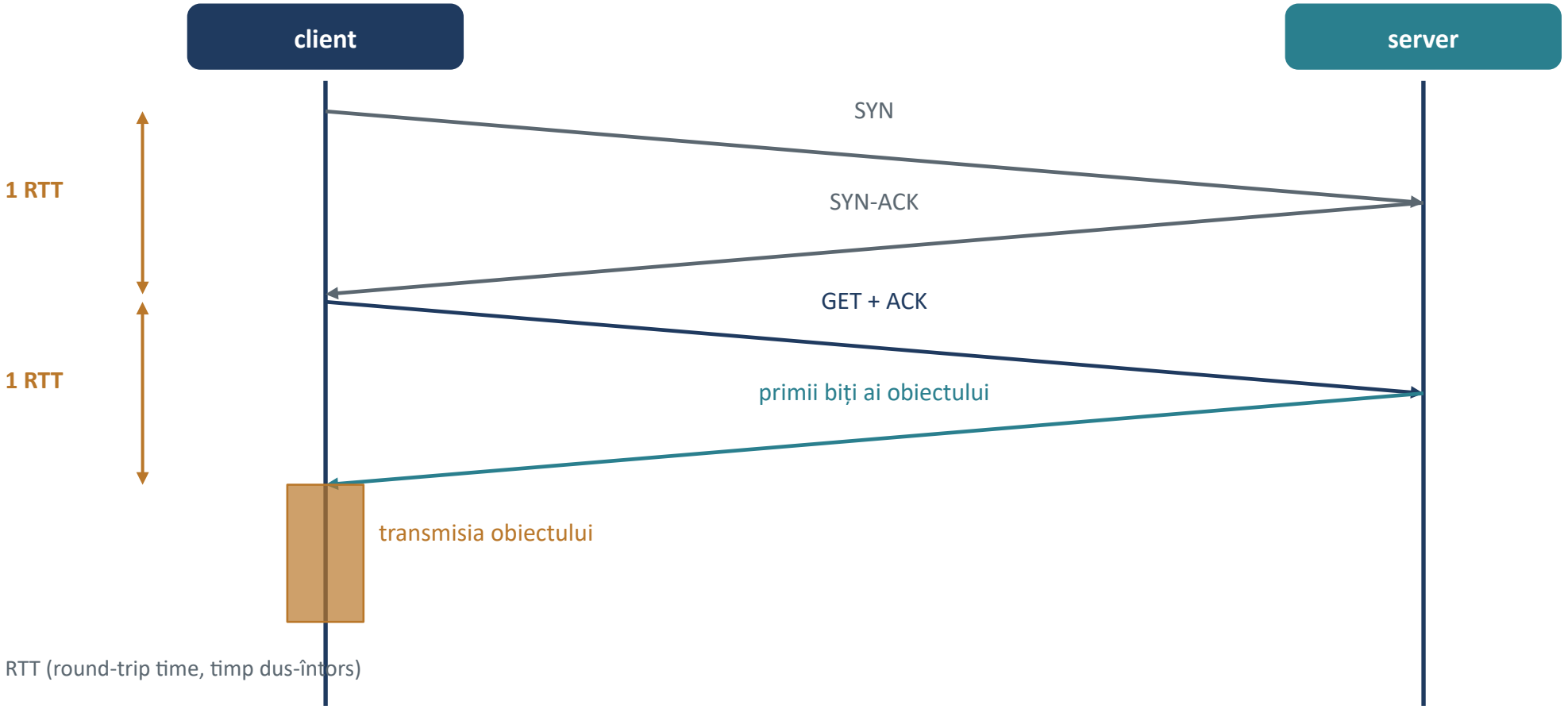
Rulează peste TCP (Transmission Control Protocol) (HTTP/1.1, HTTP/2) sau peste QUIC (HTTP/3). HTTP (HyperText Transfer Protocol). Starea (autentificări, coșuri) se adaugă cu **cookie-uri** — săptămâna viitoare.

fără stare (*stateless*)
nicio amintire a cererilor anterioare

Ideea-cheie HTTP este un protocol cerere–răspuns fără stare: servere mai simple și mai robuste.

QUIC (protocol de transport peste UDP, User Datagram Protocol; numele inițial: Quick UDP Internet Connections)

7 · Un obiect, nepersistent: 2 RTT + transmisie

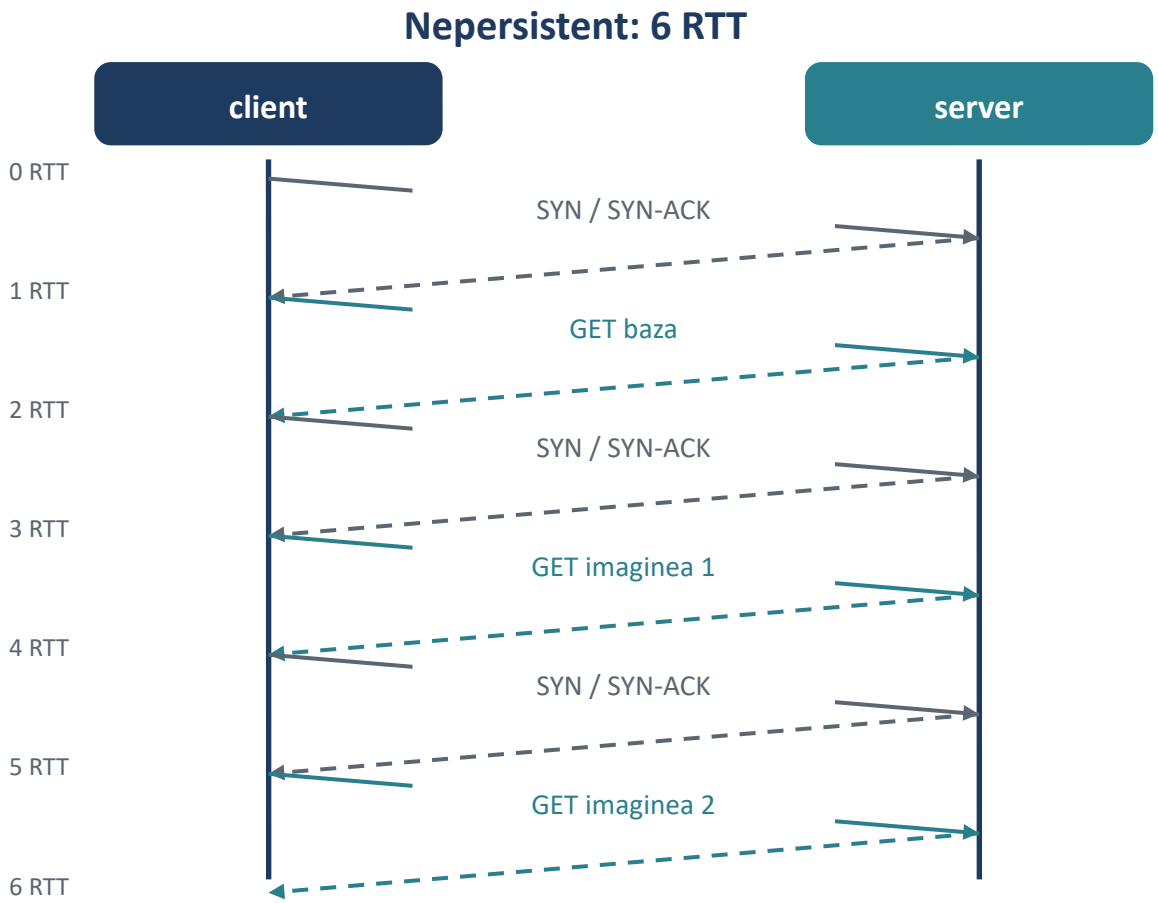


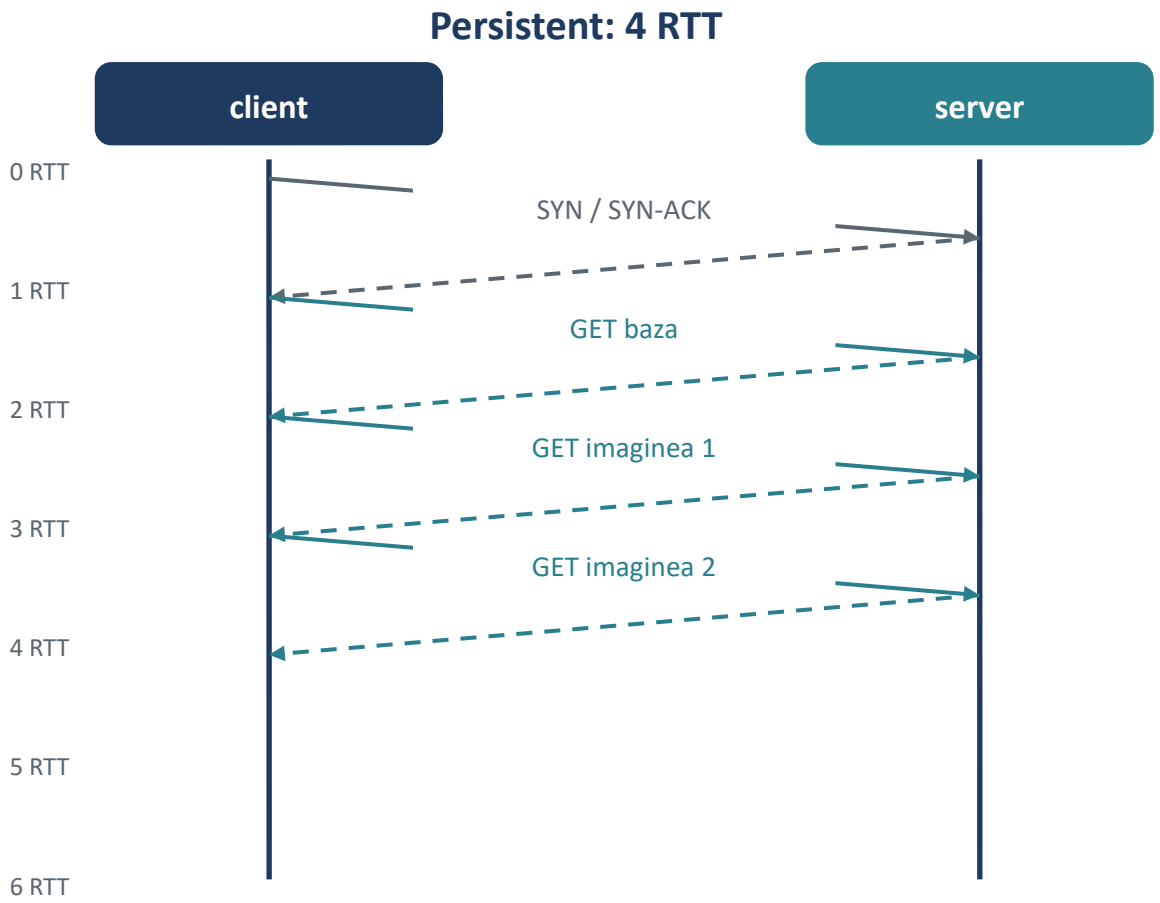
timp per obiect
2 RTT
+ transmisie

Ideea-cheie HTTP nepersistent plătește o stabilire de conexiune TCP (Transmission Control Protocol) pentru fiecare obiect: câte 2 RTT.

HTTP (HyperText Transfer Protocol) · SYN (synchronize, sincronizare) · ACK (acknowledgement, confirmare)

7 · Nepersistent vs persistent: pagina de bază + 2 imagini



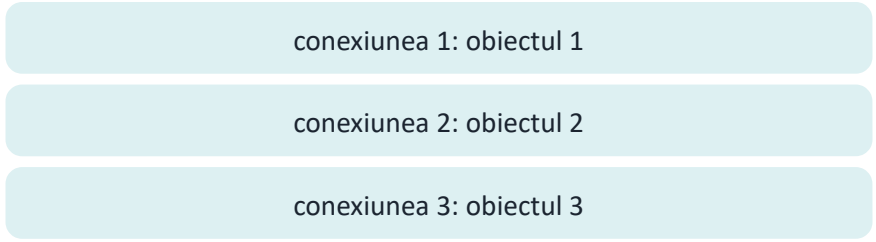


Ideea-cheie Conexiunile persistente (implicite în HTTP/1.1) plătesc stabilirea conexiunii o singură dată: 1 RTT per obiect după primul.

HTTP (HyperText Transfer Protocol) · RTT (round-trip time, timp dus-întors) · SYN (synchronize, sincronizare) · ACK (acknowledgement, confirmare)

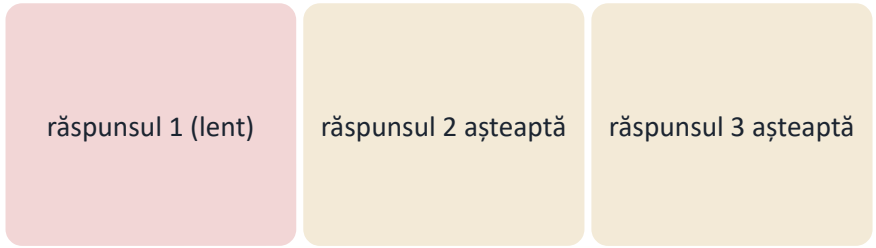
7 · Mai multe lucruri deodată

Conexiuni paralele
mai multe conexiuni TCP deodată (≈ 6 per server în HTTP/1.1)



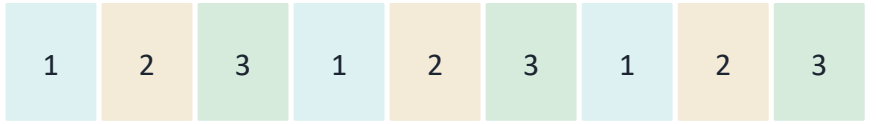
ascund latența · mai multă încărcare pe server · inechitabile față de alții

Pipelining (HTTP/1.1)
mai multe cereri pe o conexiune; răspunsurile **în ordine**



un răspuns lent le blochează pe celelalte: **blocare la capul cozii** (*head-of-line blocking*)

Multiplexare (HTTP/2, HTTP/3)
cereri împărțite în cadre, **întrețesute** pe o conexiune



fără blocare la capul cozii la nivelul HTTP

Ideea-cheie Paralelismul ascunde timpii dus-întors; HTTP/2 și HTTP/3 o fac pe o singură conexiune, fără blocare la capul cozii.

HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol)

7 · Exemplu rezolvat: 1 pagină de bază + 10 imagini, RTT = 50 ms

Nepersistent, câte unul

11 obiecte × 2 RTT = 22 RTT

Nepersistent, 5 în paralel

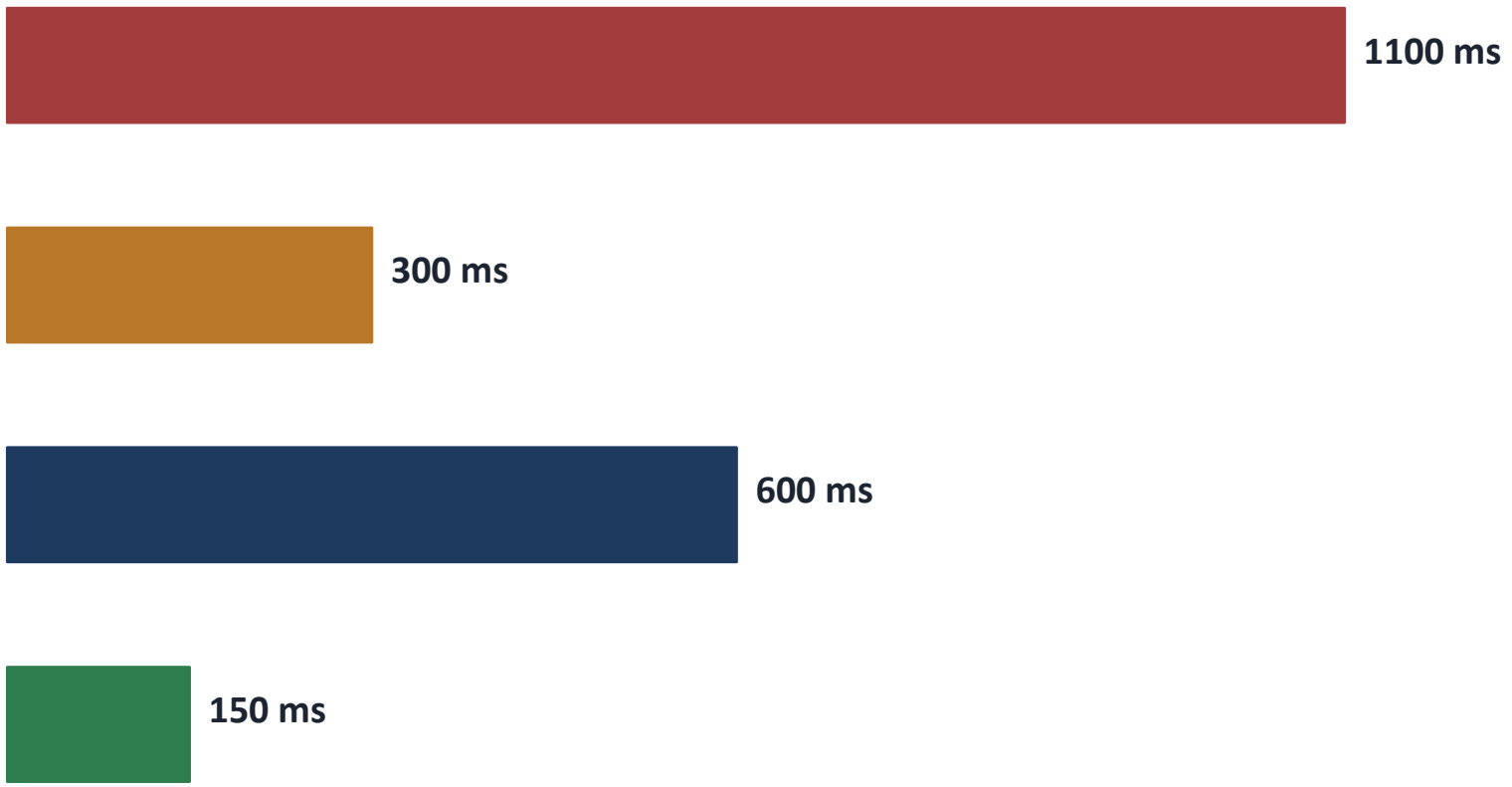
2 RTT + 2 runde × 2 RTT = 6 RTT

Persistent, câte o cerere

2 RTT + 10 × 1 RTT = 12 RTT

Persistent + pipelining / HTTP/2

2 RTT + 1 RTT = 3 RTT



Ideea-cheie La paginile cu multe obiecte mici, numărul de timpi dus-întors — nu rata legăturii — stabilește timpul de încărcare.

HTTP (HyperText Transfer Protocol) · RTT (round-trip time, timp dus-întors)

8 · Anatomia unei cereri HTTP/1.1

```
GET /courses/networks/week3.html HTTP/1.1
Host: www.example.org
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
Accept: text/html
Connection: keep-alive
(linie goală)
```

1. **Linia de cerere** (*request line*): metodă · țintă (cale și interogare) · versiune.
2. **Host** este obligatoriu: o adresă IP poate găzdui mii de site-uri.
3. **Linii de antet** (*header lines*): `nume: valoare`, câte una pe linie.
4. **Linia goală** (doar CR LF (carriage return, line feed — retur de car, linie nouă)) încheie antetul; poate urma un **corp** (*body*), de ex. datele unui formular.

Specificată în RFC (Request for Comments) 9110 (semantica HTTP) și RFC 9112 (mesajele HTTP/1.1), 2022.

Ideea-cheie O cerere înseamnă linii de text: linia de cerere, antetele, linia goală, un corp opțional.

HTTP (HyperText Transfer Protocol) · X11 (X Window System, versiunea 11) · IP (Internet Protocol) · CR (carriage return, retur de car) · LF (line feed, linie nouă)

8 · Metode: sigure și idempotente

	sigură + idempotentă	sigură, neidempotentă
sigure (*safe*)	**GET** aduce **HEAD** doar antetul **OPTIONS** ce metode?	*(nicio metodă standard)*
nesigure	**idempotentă, nesigură** **PUT** creează sau înlocuiește **DELETE** șterge	**niciuna** **POST** trimite, de ex. o comandă **PATCH** modifică o parte

Ideea-cheie După un răspuns pierdut, clientul poate repeta GET, PUT sau DELETE; repetarea unui POST poate comanda de două ori.

8 · Anatomia unui răspuns HTTP/1.1

```
HTTP/1.1 200 OK
Last-Modified: Fri, 16 Oct 2026 14:12:30 GMT
Content-Type: text/html
Content-Length: 5120
(linie goală)
<!doctype html> … (5120 de octeți de corp)
```

1. **Linia de stare** (*status line*): versiune · cod de stare · motiv.
2. **Antetele** descriu corpul (tip, dimensiune, dată). GMT = Greenwich Mean Time.
3. **Content-Length** spune exact câți octeți de corp urmează.
4. **Corpul**: obiectul însuși, după linia goală.

Ideea-cheie Răspunsul oglindește cererea: o primă linie, antete, linie goală, apoi exact Content-Length octeți.

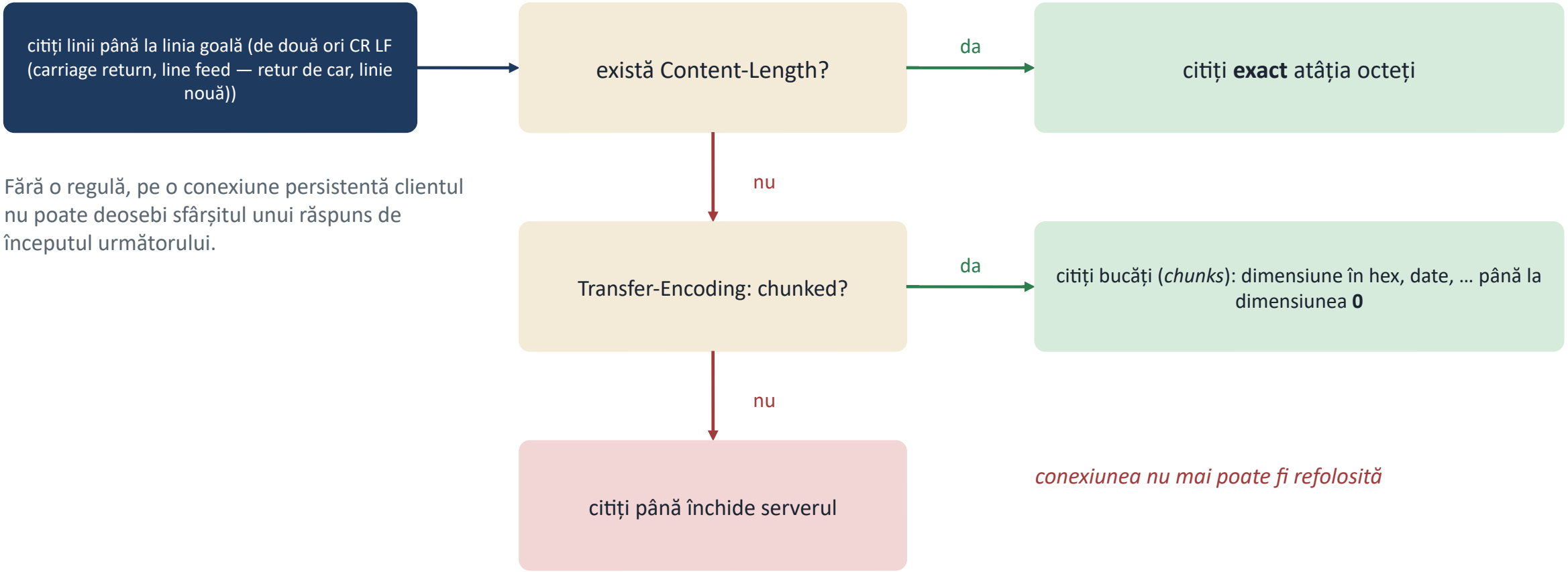
HTTP (HyperText Transfer Protocol)

8 · Coduri de stare: cinci clase

1xx	**informare**	100 Continue · 101 Switching Protocols
2xx	**succes**	200 OK · 201 Created · 204 No Content
3xx	**redirecționare: căutați în altă parte**	301 Moved Permanently · 302 Found · 304 Not Modified
4xx	**eroare a clientului: corectați cererea**	400 Bad Request · 401 Unauthorized · 403 Forbidden · 404 Not Found · 405 Method Not Allowed
5xx	**eroare a serverului: nu e vina voastră**	500 Internal Server Error · 502 Bad Gateway · 503 Service Unavailable

Ideea-cheie 2xx: gata · 3xx: mergeți în altă parte · 4xx: a greșit clientul · 5xx: a eșuat serverul.

8 · Unde se termină un mesaj? Regulile de încadrare



Fără o regulă, pe o conexiune persistentă clientul nu poate deosebi sfârșitul unui răspuns de începutul următorului.

Ideea-cheie Orice protocol peste TCP trebuie să rezolve încadrarea; HTTP/1.1 folosește o linie goală, o lungime sau bucăți.

HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol) · CR (carriage return, retur de car) · LF (line feed, linie nouă)

8 · Codarea chunked, decodificată

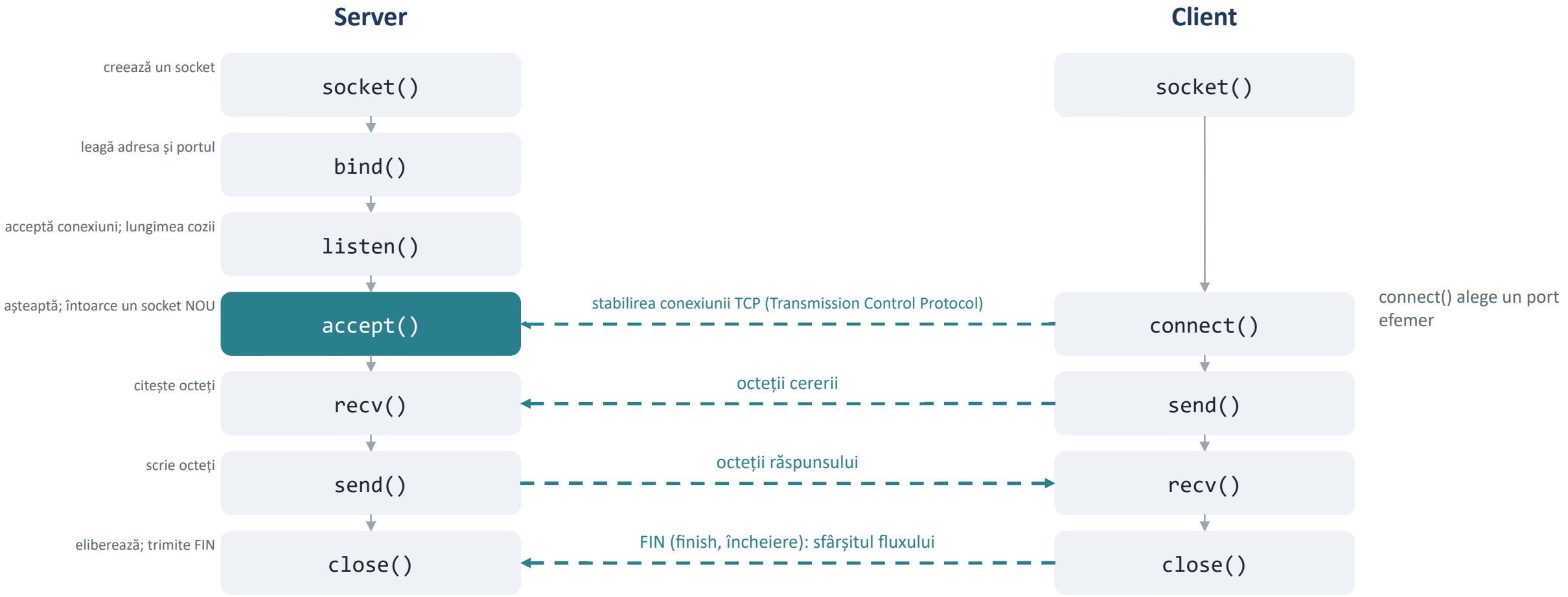


1 Fiecare lungime și fiecare bucată de date sunt urmate de CR LF (carriage return, line feed — retur de car, linie nouă). Se folosește când serverul nu știe dinainte lungimea (o pagină generată chiar în timp ce este trimisă).

Ideea-cheie Codarea chunked permite unui server să trimită în flux un corp de lungime necunoscută și să păstreze totuși conexiunea.

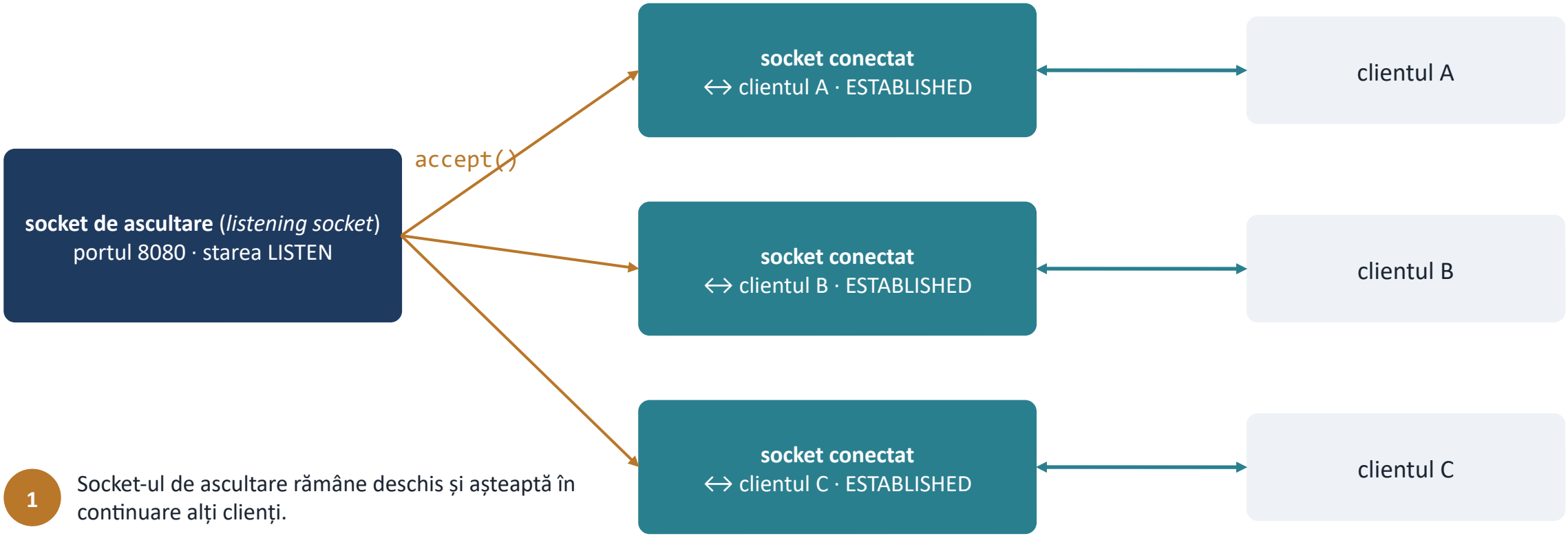
CR (carriage return, retur de car) · LF (line feed, linie nouă)

9 · Apelurile socket ale unui server și ale unui client TCP



Ideea-cheie Server: socket → bind → listen → accept. Client: socket → connect. Amândoi: send, recv, close.

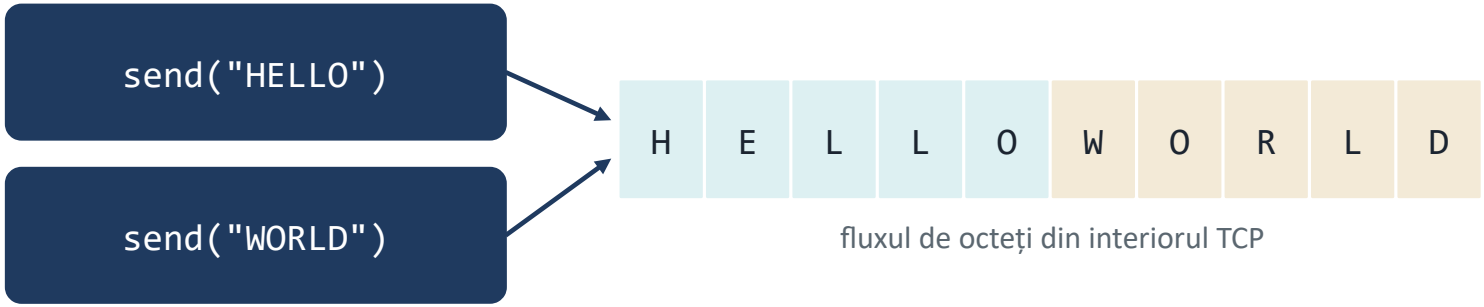
9 · accept() întoarce un socket nou



1 Socket-ul de ascultare rămâne deschis și așteaptă în continuare alți clienți.

Ideea-cheie Un socket de ascultare pentru fiecare serviciu; un socket conectat pentru fiecare client.

9 · Un flux nu are granițe între mesaje



posibil: recv → 10

HELLOWORLD

posibil: recv → 5, recv → 5

HELLO WORLD

posibil: recv → 3, recv → 7

HEL LOWORLD

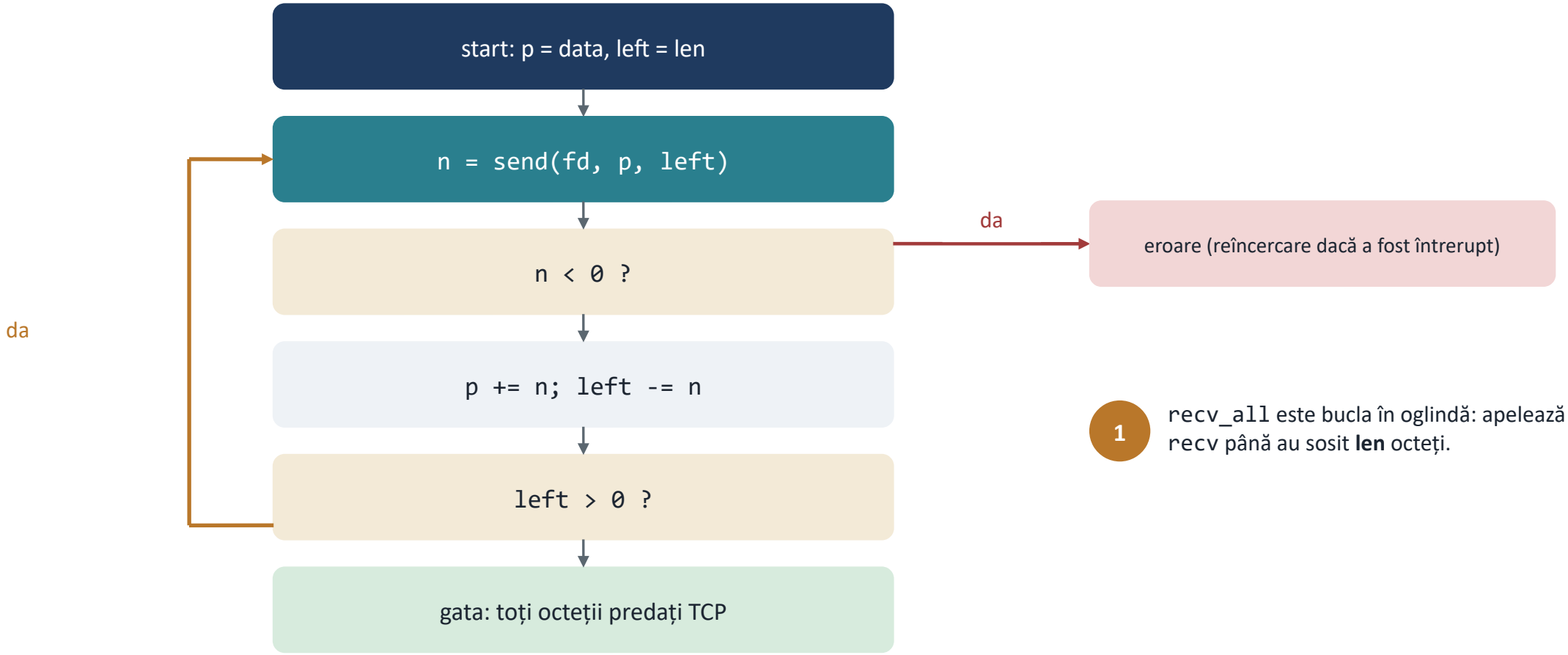
1 Niciodată posibil: octeți pierduți sau **reordonați**.

2 Așadar un program trebuie să ruleze **în buclă** până are ce așteaptă, iar protocolul trebuie să spună unde se termină un mesaj.

Ideea-cheie TCP păstrează octeții și ordinea lor — nu și granițele dintre send-urile voastre.

TCP (Transmission Control Protocol)

9 · send() poate accepta mai puțini octeți: bucla send_all



1 `recv_all` este bucla în oglindă: apelează `recv` până au sosit **len** octeți.

Ideea-cheie Învelește mereu send într-o buclă (send_all; sendall în Python), iar recv într-o buclă când ai nevoie de un număr exact.

TCP (Transmission Control Protocol) · fd (file descriptor, descriptor de fișier)

9 · Ordinea octeților: htons(8080)

8080 = **0x1F90**

memoria unei mașini x86 (little-endian)

90	1F



htons() (host to network short)

ordinea de rețea a octeților (big-endian)

1F	90

Dacă uitați htons:
octeții 90 1F pleacă așa cum sunt
și sunt citiți ca **0x901F = 36 895**
→ serverul ascultă pe portul 36 895

Cele patru funcții de conversie
htons · htonl: gazdă → rețea (short = 16 biți, long = 32 de biți)
ntohs · ntohl: rețea → gazdă

Ideea-cheie Convertiți orice port, adresă și întreg binar în ordinea de rețea înainte de a-l trimite.

htons (host to network short, gazdă → rețea, 16 biți) · htonl (host to network long, gazdă → rețea, 32 de biți) · ntohl (network to host long, rețea → gazdă, 32 de biți) · ntohs (network to host short, rețea → gazdă, 16 biți)

Rezumatul Cursului 3

Arhitecturi
client–server, de la egal la egal, hibrid

Încadrare
linie goală, Content-Length, chunked; apelurile socket

Socket-uri
proces = adresă IP + port; conexiune = cvadruplu

Timpi dus-întors
2 RTT per obiect nepersistent, 1 RTT persistent

Săptămâna 3

Nevoi de transport
fiabilitate, debit, temporizare, securitate

Web-ul
obiecte, URL-uri, HTTP fără stare

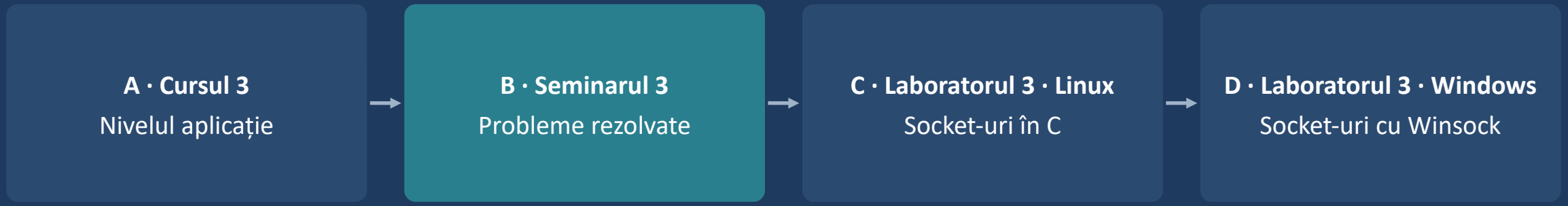
TCP vs UDP
flux fiabil vs datagrame simple; TLS adaugă securitate

Protocol
tipuri, sintaxă, semantică, reguli

Ideea-cheie De la arhitecturi la octeții dintr-un socket: tot ce îi trebuie săptămâna aceasta unui programator de aplicații.

HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · TLS (Transport Layer Security) · URL (Uniform Resource Locator) · RTT (round-trip time, timp dus-întors) · IP (Internet Protocol)

Partea B · Seminarul 3: probleme rezolvate



În această parte

Problema 1 · Ce transport?

Problema 4 · Citirea unui schimb HTTP

Problemele 7–8 · Coduri de stare, repetări

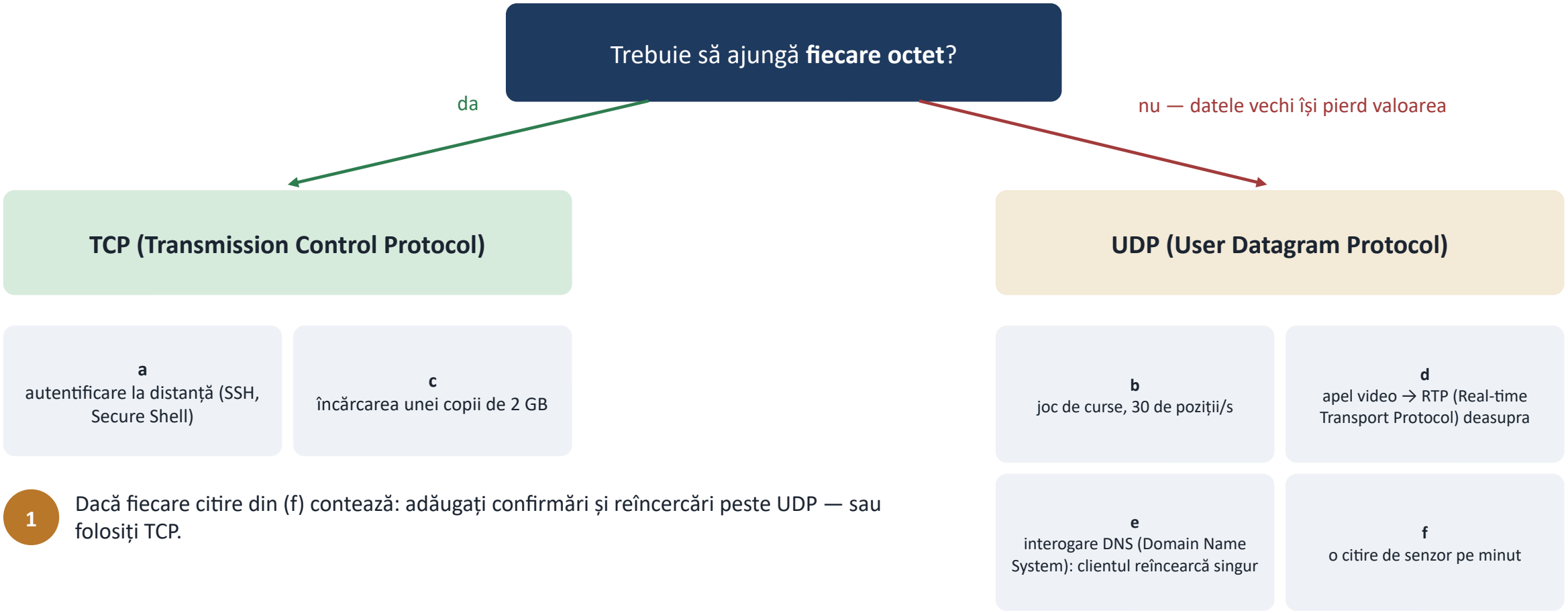
Problemele 2–3 · Timpi de încărcare

Problemele 5–6 · Încadrare

Problemele 9–10 · Socket-uri și o proiectare

HTTP (HyperText Transfer Protocol)

Problema 1 · Alegerea transportului: un arbore de decizie



1 Dacă fiecare citire din (f) contează: adăugați confirmări și reîncercări peste UDP — sau folosiți TCP.

Ideea-cheie Nu tolerează pierderi și e elastic → TCP. Sensibil la timp și tolerant la pierderi → UDP, plus doar ce îi trebuie aplicației.

Problema 2 · Timpul de încărcare: pagina de bază + 6 imagini

Baza 50 KB · 6 imagini × 100 KB · RTT 40 ms · legătura de acces 20 Mbit/s → baza 20 ms, fiecare imagine 40 ms de transmisie

Nepersistent, câte unul
baza 2·40+20; 6 × (2·40+40)



820 ms

Persistent, fără pipelining
baza 100; 6 × (40+40)



580 ms

Persistent + pipelining
baza 100; 40 + 6 × 40



380 ms

RTT (round-trip time, timp dus-întors)

timpi dus-întors (RTT)

transmisie

1 Transmisia imaginilor (240 ms) nu se micșorează niciodată; doar timpii dus-întors (bleumarin) pot fi eliminați.

Ideea-cheie Persistența elimină stabilirile de conexiune, pipelining-ul elimină așteptarea — timpul de transmisie rămâne.

Problema 2.5 · Aceeași pagină cu 60 de imagini de 2 KB

Nepersistent
60 × (80 + 0,8) ms



Persistent, fără pipelining
60 × (40 + 0,8) ms



Pipelining
40 + 60 × 0,8 ms



RTT (round-trip time, timp dus-întors)



1 Timpi după pagina de bază. La multe obiecte mici, **numărul de timpi dus-întors** decide totul.

Ideea-cheie Multe obiecte mici: numărați timpii dus-întors, nu octeții.

Problema 3 · Șase conexiuni paralele împart o singură legătură

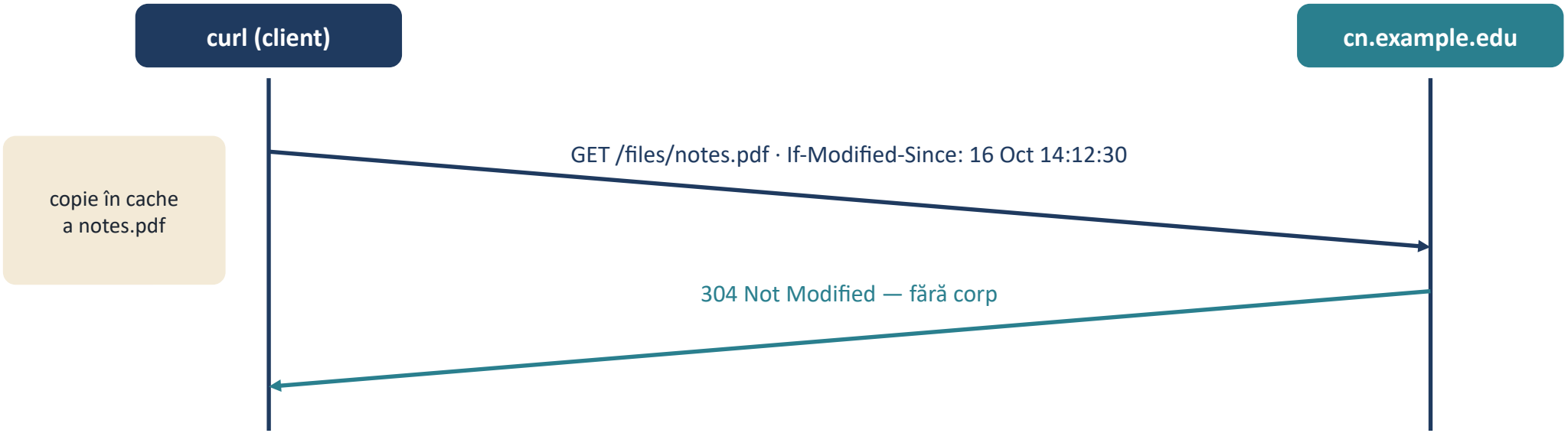


1 Nu de 6 ori mai bine decât 820 ms: paralelismul ascunde timpii dus-întors, **nu transmisia**.

Ideea-cheie Conexiunile paralele suprapun așteptarea, dar împart aceeași legătură-gâtuire (bottleneck).

RTT (round-trip time, timp dus-întors)

Problema 4 · Un GET condiționat, cu răspunsul 304



URL complet
http://cn.example.edu/files/notes.pdf

persistentă?
da: HTTP/1.1, fără Connection: close

lungimea corpului
0 — un 304 nu are niciodată corp

de ce
verifică prospețimea fără o nouă descărcare

Ideea-cheie Un GET condiționat (conditional GET) economisește transferul: 304 înseamnă „copia ta e încă bună” (cache, săptămâna viitoare).

HTTP (HyperText Transfer Protocol) · URL (Uniform Resource Locator)

Problema 5 · Decodificarea unui corp chunked

`1a`	= 26 octeți	`abcdefghijklmnopqrstuvwxyz`
`a`	= 10 octeți	`0123456789`
`0`	= 0 octeți	**sfârșitul corpului** (apoi o linie goală)

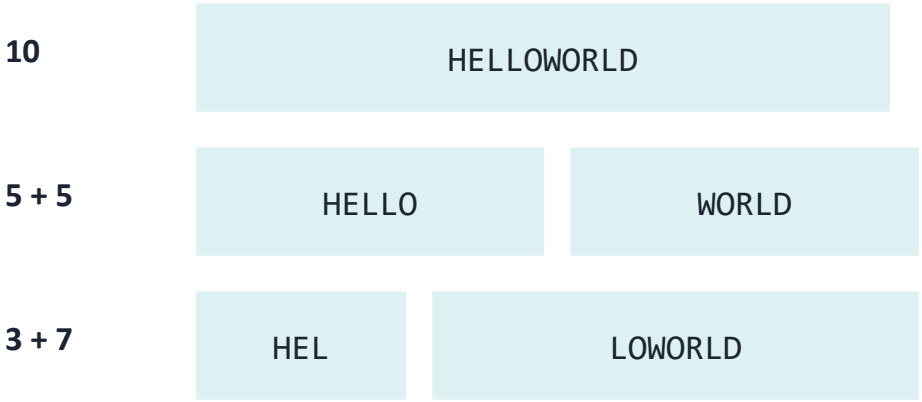
26 + 10
= 36 de octeți

1 După bucata de dimensiune 0, conexiunea este liberă pentru răspunsul **următor** — încadrarea face posibilă refolosirea.

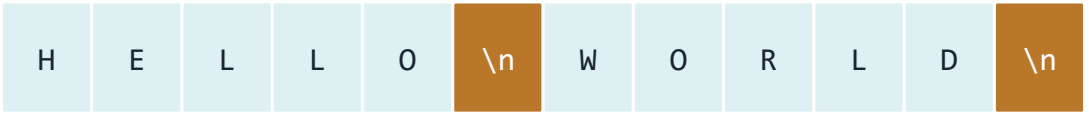
Ideea-cheie Citiți dimensiunea în hexazecimal, apoi exact atâția octeți; repetați până la dimensiunea 0.

Problema 6 · Două send-uri, multe recv-uri, două remedii

Ce poate vedea serverul



Remediul 1 · delimitator (*delimiter*, 12 octeți)



citiți până la fiecare \n — delimitatorul nu trebuie să apară niciodată în interiorul unui mesaj

Remediul 2 · prefix de lungime (*length prefix*, 18 octeți) — merge pentru orice conținut, chiar binar



lungimea pe 4 octeți, în ordinea de rețea, apoi exact atâția octeți

Ideea-cheie Octeții sosesc mereu în ordine — granițele niciodată. Marcați-le cu un delimitator sau cu o lungime.

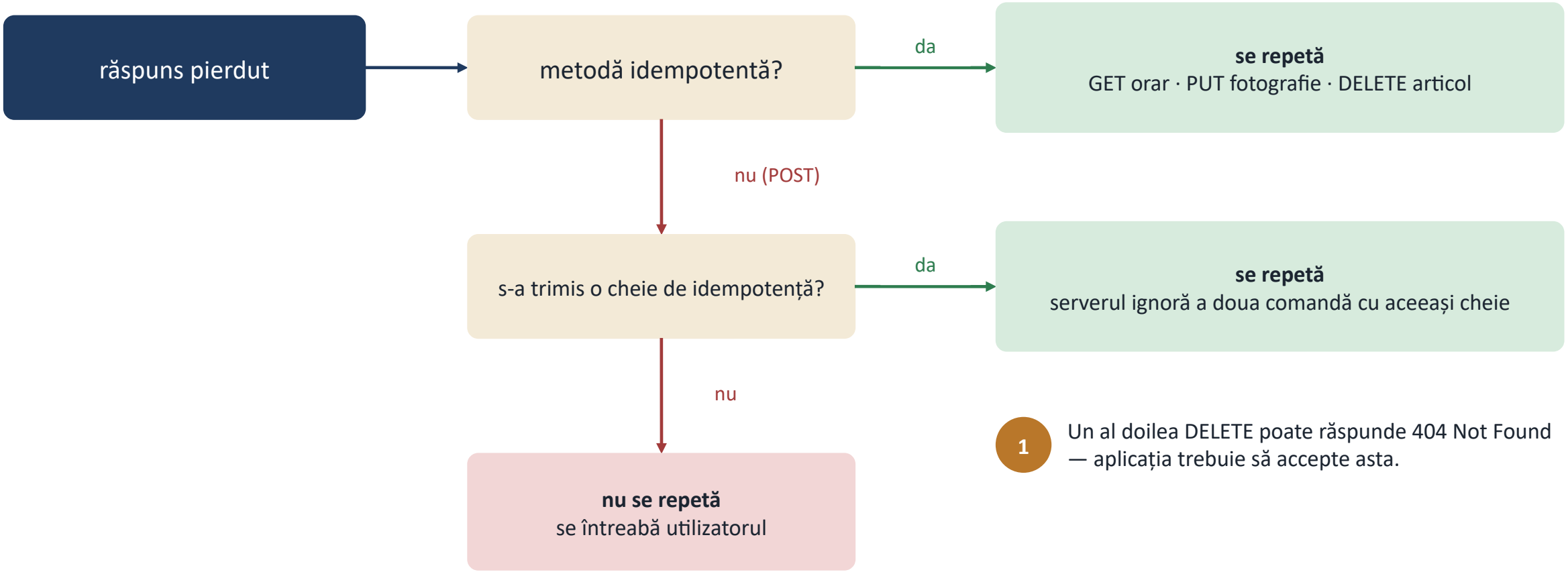
Problema 7 · Asocierea situațiilor cu codurile de stare

site-ul s-a mutat definitiv pe https://	**301 Moved Permanently**
/admin fără autentificare	**401 Unauthorized**
/admin ca student (autentificat)	**403 Forbidden**
DELETE pe un server care nu șterge niciodată	**405 Method Not Allowed**
POST a creat un cont nou	**201 Created**
aplicația s-a prăbușit	**500 Internal Server Error**
linia de cerere „GTE / HTTP/1.1”	**400 Bad Request**

Ideea-cheie 401 = cine ești? · 403 = te cunosc, dar nu ai voie · 405 = nu cu această metodă.

HTTP (HyperText Transfer Protocol)

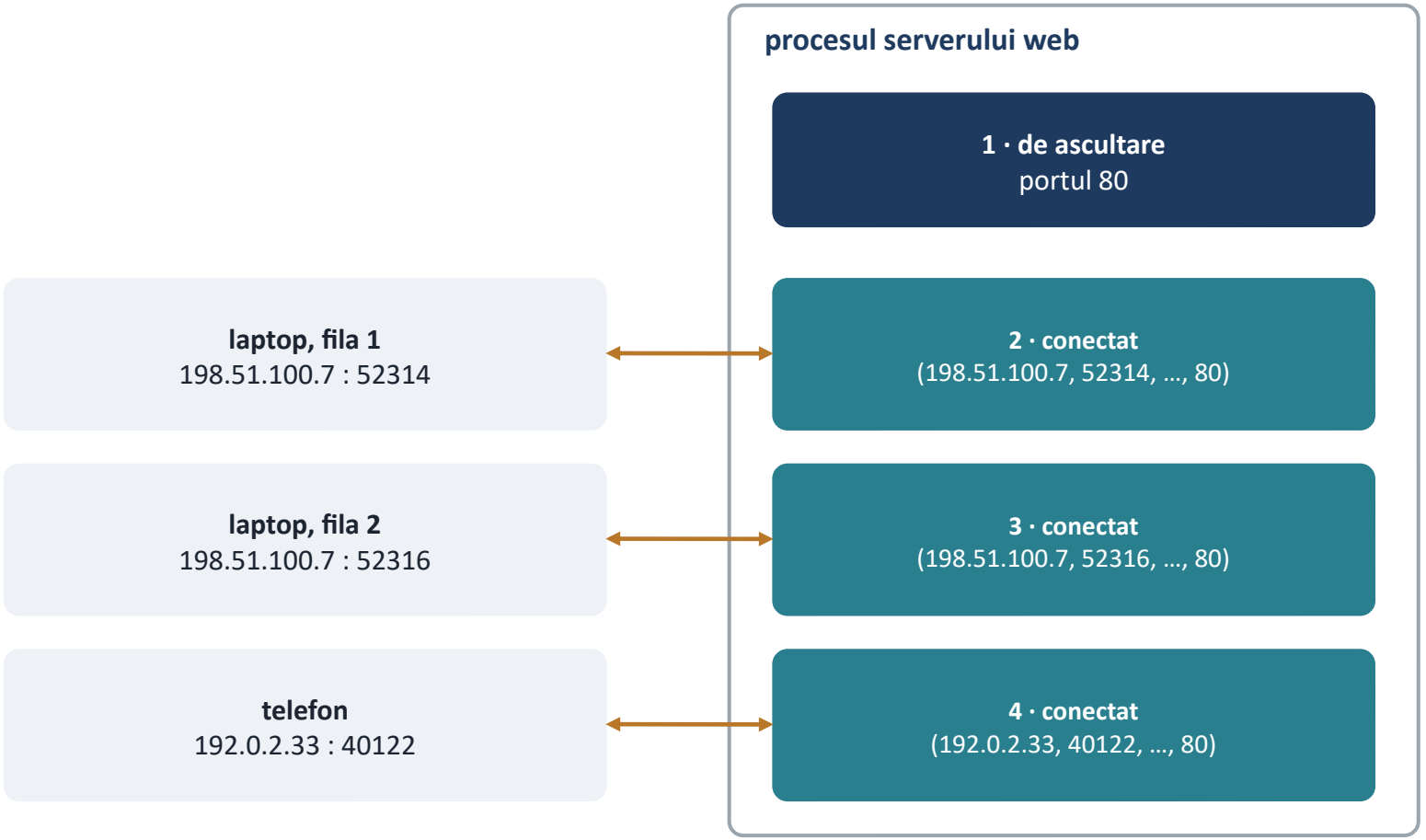
Problema 8 · Poate aplicația repeta o cerere după un răspuns pierdut?



1 Un al doilea DELETE poate răspunde 404 Not Found — aplicația trebuie să accepte asta.

Ideea-cheie Repetați liber cererile idempotente; repetați un POST doar cu o cheie pe care serverul o poate recunoaște.

Problema 9 · Socket-uri și cvadruple pe un server aglomerat



1 Cele două file ale laptopului diferă **doar prin portul sursă** — suficient ca să fie deosebite.

Ideea-cheie Socket-urile unui server = 1 de ascultare + câte 1 pentru fiecare conexiune; fiecare conexiune are un cvadruplu unic.

Problema 10 · Proiectarea unui protocol de încărcare de fișiere

Cerere

```
PUT notes.pdf 52113\r\n
```

linia de antet se termină la primul CR LF (carriage return, line feed — retur de car, linie nouă)

cei 52 113 octeți bruți ai fișierului

corpul se termină după exact <size> octeți

Răspuns

```
OK\r\n
```

```
ERR 413 too large\r\n
```

nume
fără spații, fără CR LF

mai multe cereri
pot urma pe aceeași conexiune

repetare
PUT cu același fișier este idempotent

închidere prematură
mai puțin de <size> octeți → se aruncă

Ideea-cheie Delimitator pentru antetul text (sigur: numele nu îl pot conține), lungime pentru corpul binar.

CR (carriage return, retur de car) · LF (line feed, linie nouă)

Partea C · Laboratorul 3 pe Linux: socket-uri în C

A · Cursul 3
Nivelul aplicație

→

B · Seminarul 3
Probleme rezolvate

→

C · Laboratorul 3 · Linux
Socket-uri în C

→

D · Laboratorul 3 · Windows
Socket-uri cu Winsock

În această parte

Pregătire · fișiere, make, terminale

Sarcina 1 · Un server fără cod (nc)

Sarcina 2 · Clientul de ecou

Sarcina 3 · Serverul de ecou

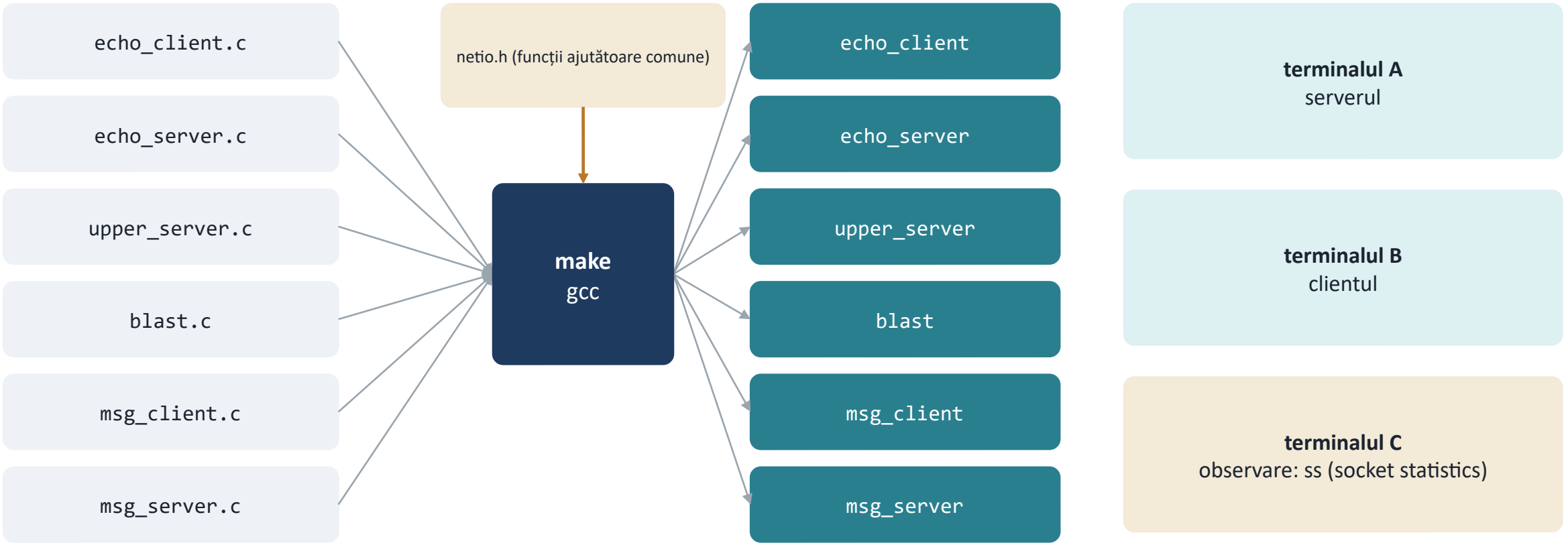
Sarcina 4 · Socket-urile văzute de sistem

Sarcina 5 · Un flux nu e un șir de mesaje

Sarcinile 6–7 · Două metode de încadrare

Sarcina 8 · Stricăm intenționat

Pregătire · De la fișierele sursă la programe care rulează



Se instalează o dată: build-essential, netcat-openbsd, manpages-dev · porturi 5000–60000

Ideea-cheie Construiți totul cu make; rulați serverul într-un terminal, clientul în altul și urmăriți din al treilea.

Sarcina 1 · Un server fără să scrieți cod: nc și ss



înainte de client

```
LISTEN  0.0.0.0:5000
```

după ce se conectează

```
ESTAB  127.0.0.1:5000 ↔ 127.0.0.1:41822
```

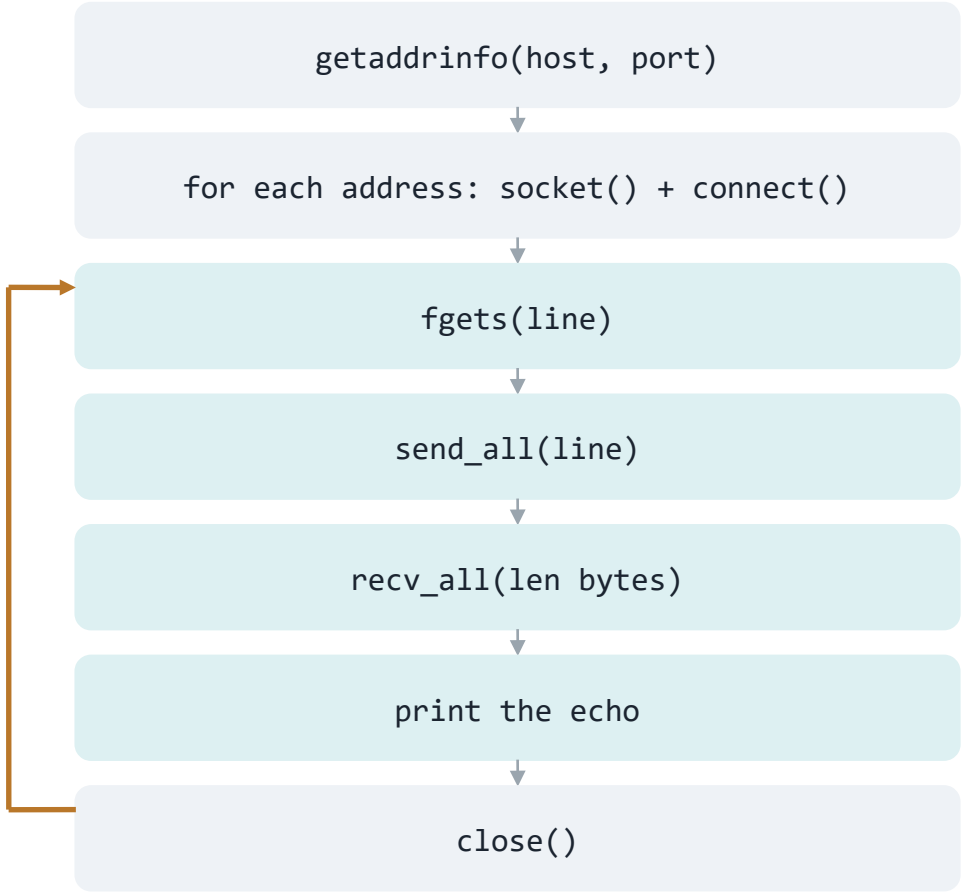
1 Q1 · portul 41822 a fost ales de sistemul de operare al clientului: un **port efemer** (*ephemeral port*).

2 Q2 · Ctrl+D în B trimite FIN (finish, încheiere); nc din A vede sfârșitul fluxului și se oprește.

Ideea-cheie Un socket de ascultare stă în LISTEN; fiecare conexiune acceptată apare ca ESTAB, cu cvadruplul ei.

ss (socket statistics)

Sarcina 2 · Clientul de ecou, pas cu pas



nume → listă de adrese (IPv6 și IPv4)

se oprește la prima care reușește

citește o linie de la tastatură

scrie toți octeții ei

citește înapoi exact tot atâția octeți

apoi linia următoare

la sfârșitul intrării (Ctrl+D)

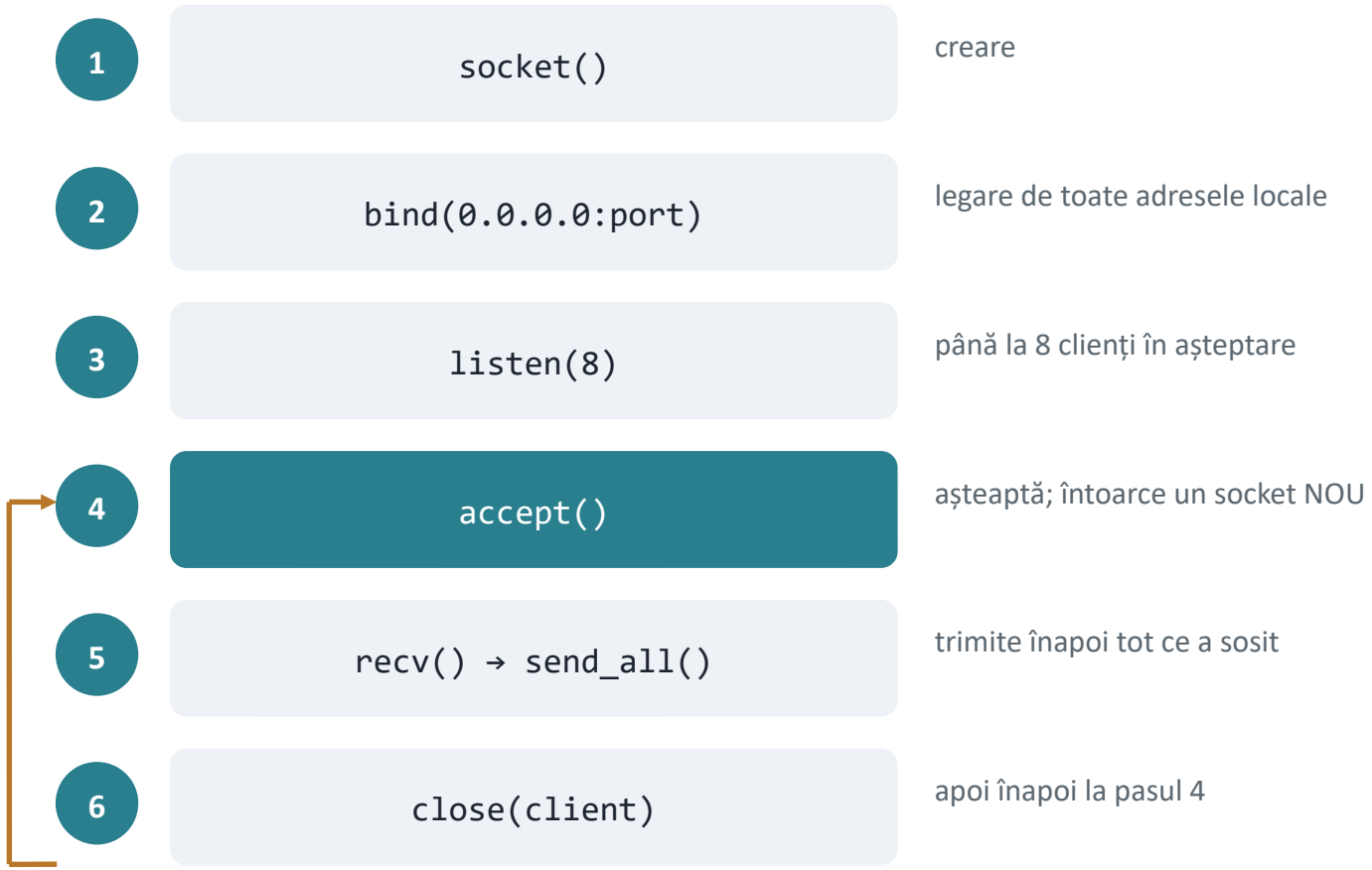
IPv4 (Internet Protocol version 4) · IPv6 (Internet Protocol version 6)

1 Q3 · un singur `recv` poate întoarce doar o **parte** din ecou; `recv_all` repetă până au sosit toți cei **len** octeți.

2 Q4 · nimic nu ascultă → „Connection refused”: gazda a răspuns la SYN (synchronize, sincronizare) cu RST (reset, resetare).

Ideea-cheie Un client robust traduce numele, încearcă fiecare adresă și citește numere exacte de octeți.

Sarcina 3 · Serverul de ecou: șase pași



SO_REUSEADDR (opțiune de socket: refolosirea adresei)
permite repornirea imediată a serverului

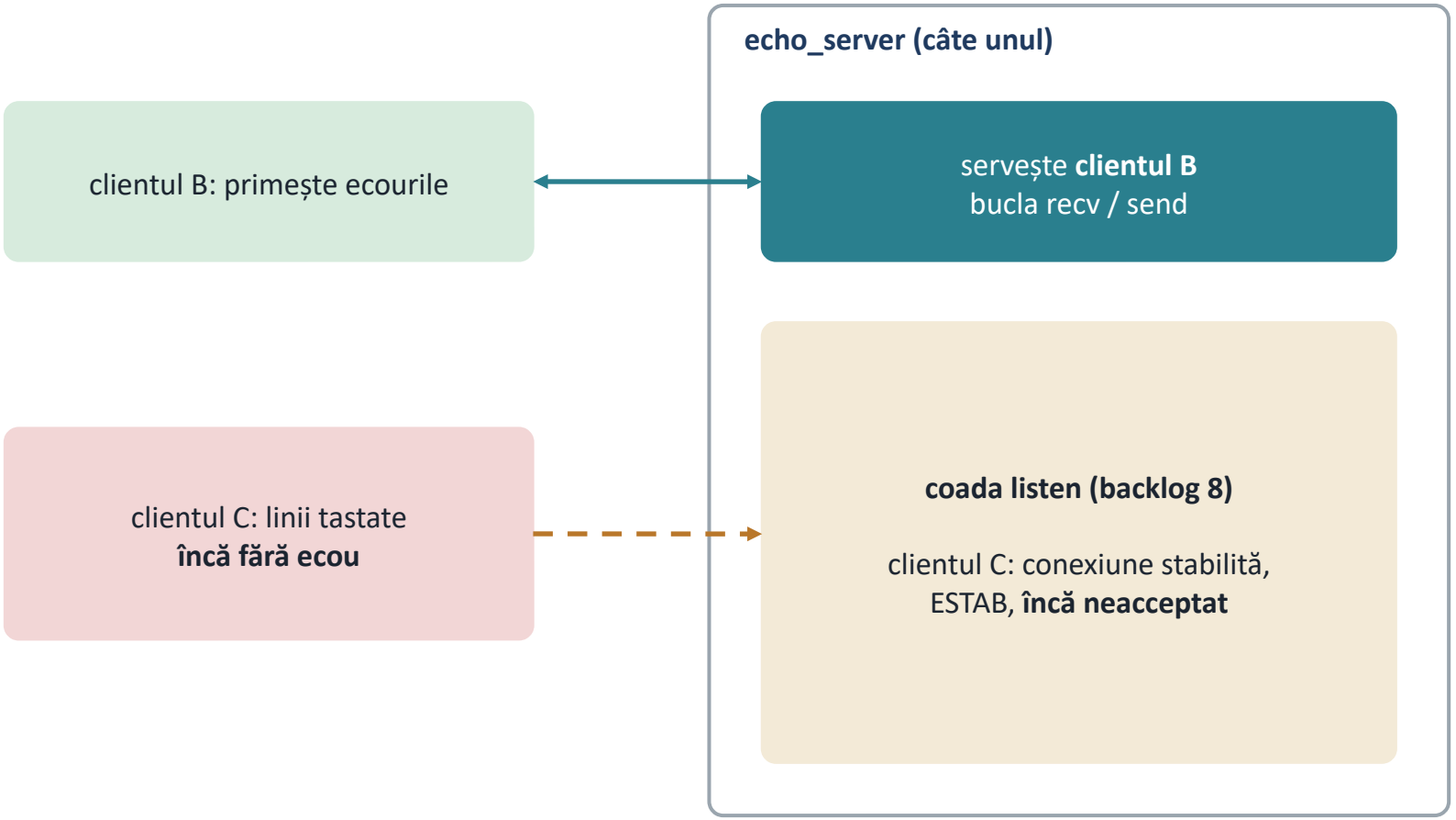
htons(port) (host to network short)
portul în ordinea de rețea a octeților

1 Q5 · serverul afișează adresa și portul **efemer** al fiecărui client, luate din accept().

Ideea-cheie socket → bind → listen → accept, apoi servește clientul până când recv întoarce 0, close și din nou accept.

htons (host to network short, gazdă → rețea, 16 biți)

Sarcina 3 · De ce așteaptă al doilea client



1. Q6 · ss -tn îl arată pe C ca ESTAB: nucleul a încheiat stabilirea conexiunii.
2. În Laboratorul 5 construim servere **concurente**.

Ideea-cheie Nucleul acceptă conexiunile într-o coadă; un server iterativ le scoate de acolo pe rând.

ss (socket statistics)

Sarcina 3 · Q7 · Dacă uitați htons

5000 = **0x1388**

memorat ca (little-endian)

88	13

citit în ordinea de rețea

88	13

→ **0x8813 = 34 835**

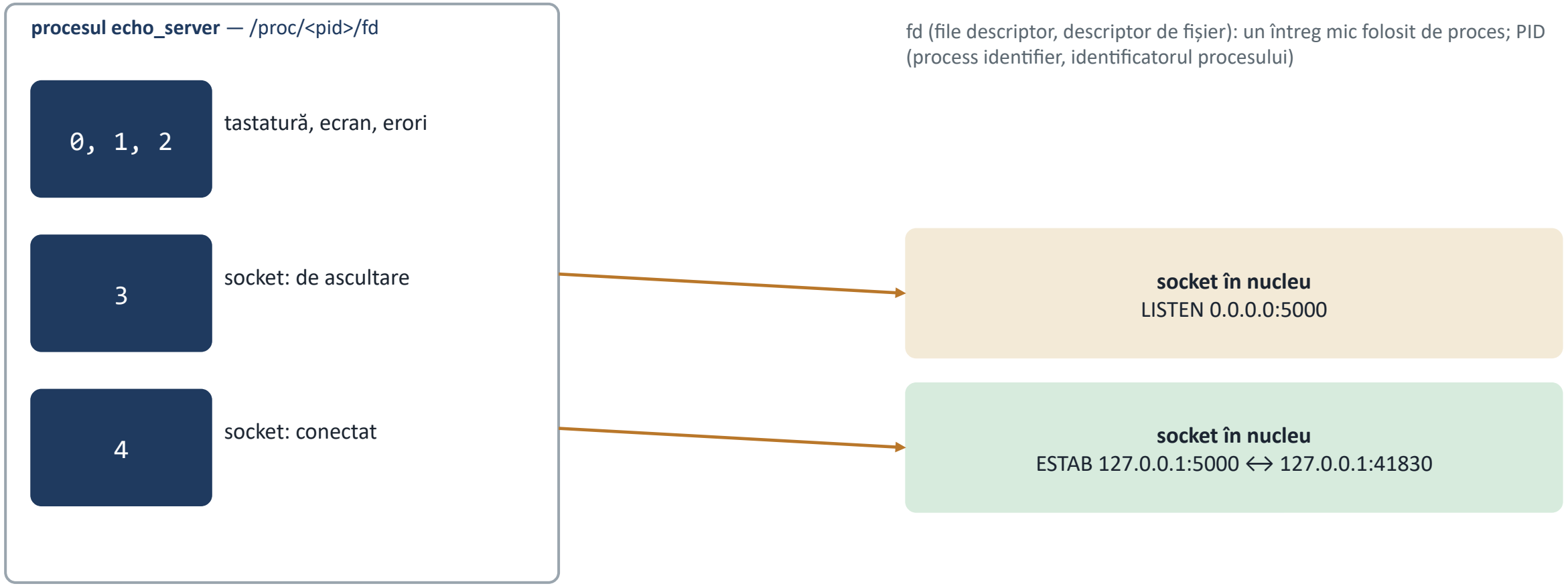
`ss -tln` arată serverul ascultând pe **34835**

un client care se conectează la 5000 primește „Connection refused”

Ideea-cheie htons() (host to network short) nu este opțional: fără el serverul ascultă pe alt port.

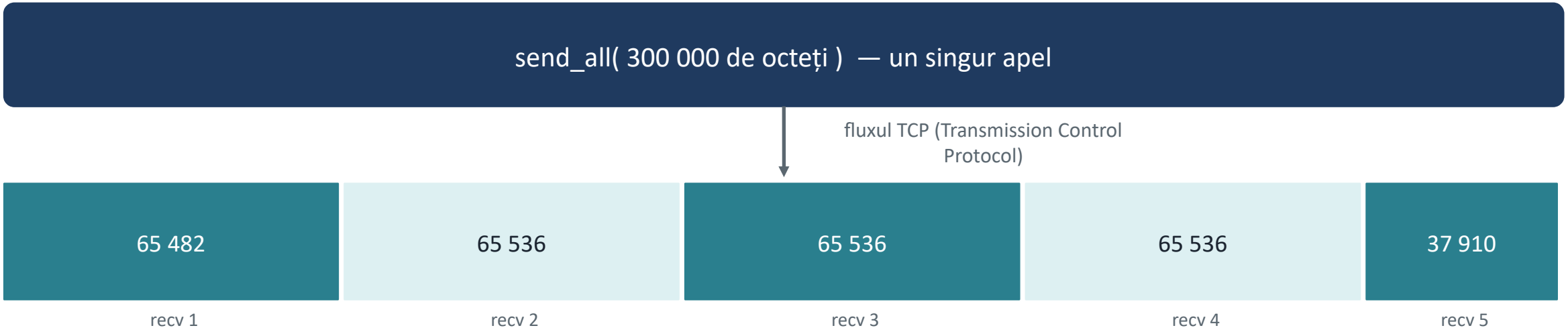
htons (host to network short, gazdă → rețea, 16 biți) · ss (socket statistics)

Sarcina 4 · Socket-urile văzute de sistemul de operare



Ideea-cheie Q8 · Cu un client: două socket-uri, fd 3 (LISTEN) și fd 4 (ESTAB).

Sarcina 5 · Un send de 300 000 de octeți, cinci recv-uri



1 Q9 · un send, **mai multe** recv-uri; dimensiunile se schimbă de la o rulare la alta și sunt limitate de memoria tampon a serverului (65 536 de octeți).

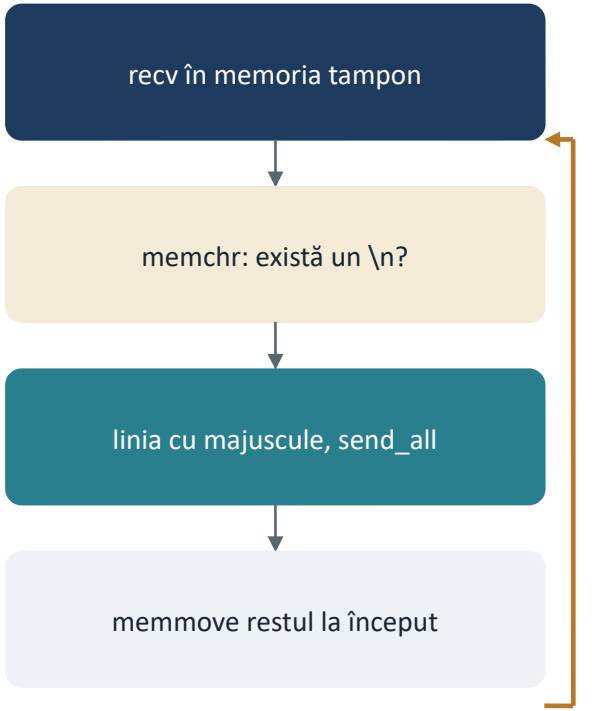
2 Q10 · cu o memorie tampon de 7 octeți, „hello world\n” sosește ca 7 + 5 — iar clientul îl primește totuși întreg, datorită lui recv_all.

Ideea-cheie Nu presupuneți niciodată că un recv = un send: repetați până aveți octeții pe care îi așteaptă protocolul.

Sarcina 6 · Încadrare pe linii: serverul de majuscule

Ce poate conține un recv()

o parte dintr-o linie	`ab`	→ așteaptă mai mult
exact o linie	`abc\n`	→ răspunde ABC
mai multe linii	`abc\n` `def\n` `gh`	→ răspunde la două linii, păstrează „gh”

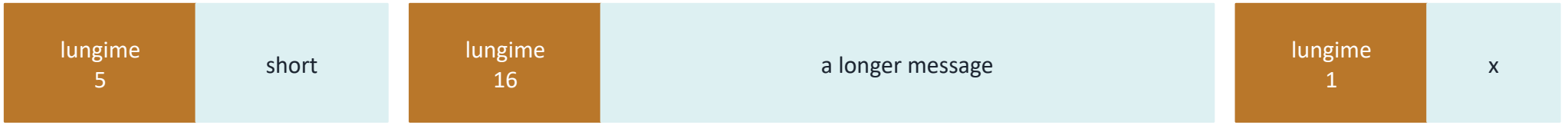


1 Q11 · „ghi” nu are \n: nu este niciodată o linie completă, deci nu primește răspuns.

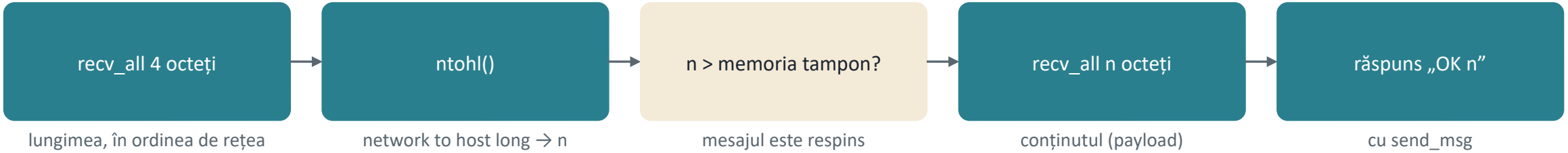
2 Q12 · o linie > 4096 de octeți umple memoria tampon: serverul răspunde ERR și închide.

Ideea-cheie Un protocol pe linii trebuie să memoreze: o linie poate fi împărțită între recv-uri sau mai multe linii pot sosi împreună.

Sarcina 7 · Încadrare cu prefix de lungime



pe fir: 4 + 5 + 4 + 16 + 4 + 1 = **34 de octeți**



1 Q13 · htonl() (host to network long) face lungimea lizibilă pe orice mașină.

2 Q14 · un fișier de peste 1 MB depășește memoria tampon a serverului: recv_msg îl respinge.

Ideea-cheie Întâi lungimea, apoi conținutul: o încadrare care funcționează pentru orice date, inclusiv binare.

htonl (host to network long, gazdă → rețea, 32 de biți) · ntohl (network to host long, rețea → gazdă, 32 de biți)

Sarcina 8 · Stricăm lucrurile intenționat

1 Conexiune refuzată

clientul trimite SYN (synchronize, sincronizare) → gazda răspunde RST (reset, resetare)

nimic nu ascultă pe acel port

2 Adresă deja folosită

bind() eșuează timp de aproximativ un minut

vechea conexiune este în TIME_WAIT; SO_REUSEADDR (opțiune de socket: refolosirea adresei) evită asta

3 Cealaltă parte a închis

clientul trimite FIN (finish, încheiere) → recv() întoarce 0

sfârșitul normal al unui flux

4 Conductă întreruptă (broken pipe)

send() într-o conexiune închisă

SIGPIPE (semnal: conductă întreruptă — broken pipe) oprește programul; dacă e ignorat, send eșuează cu EPIPE (eroare: conductă întreruptă — broken pipe)

Ideea-cheie Q15–Q16 · Fiecare mesaj de eroare numește un eveniment TCP: RST, TIME_WAIT, FIN sau o scriere într-o conexiune moartă.

TCP (Transmission Control Protocol)

Aceleași programe în Python

C	Python
`socket(AF_INET, SOCK_STREAM)`	`socket.socket(AF_INET, SOCK_STREAM)`
`bind() + listen(8)`	`ls.bind((…)); ls.listen(8)`
`accept()`	`conn, addr = ls.accept()`
`recv() în buclă`	`while data := conn.recv(65536)`
`send_all()`	`conn.sendall(data)`
`close()`	`with … : (închide automat)`

Ideea-cheie Aceleași apeluri, sintaxă mai scurtă: comportamentul fluxului — și nevoia de buclă — sunt identice.

SOCK_STREAM (socket de tip flux de octeți) · AF_INET (address family Internet, familia de adrese Internet)

Partea D · Laboratorul 3 pe Windows: Winsock

A · Cursul 3
Nivelul aplicație

→

B · Seminarul 3
Probleme rezolvate

→

C · Laboratorul 3 · Linux
Socket-uri în C

→

D · Laboratorul 3 · Windows
Socket-uri cu Winsock

În această parte

Instrumente · compilare cu ws2_32

Sarcina 1 · Un socket de ascultare în PowerShell

Sarcina 4 · Socket-urile văzute de Windows

Sarcinile 6–7 · Încadrarea, portată

Winsock vs POSIX: diferențele

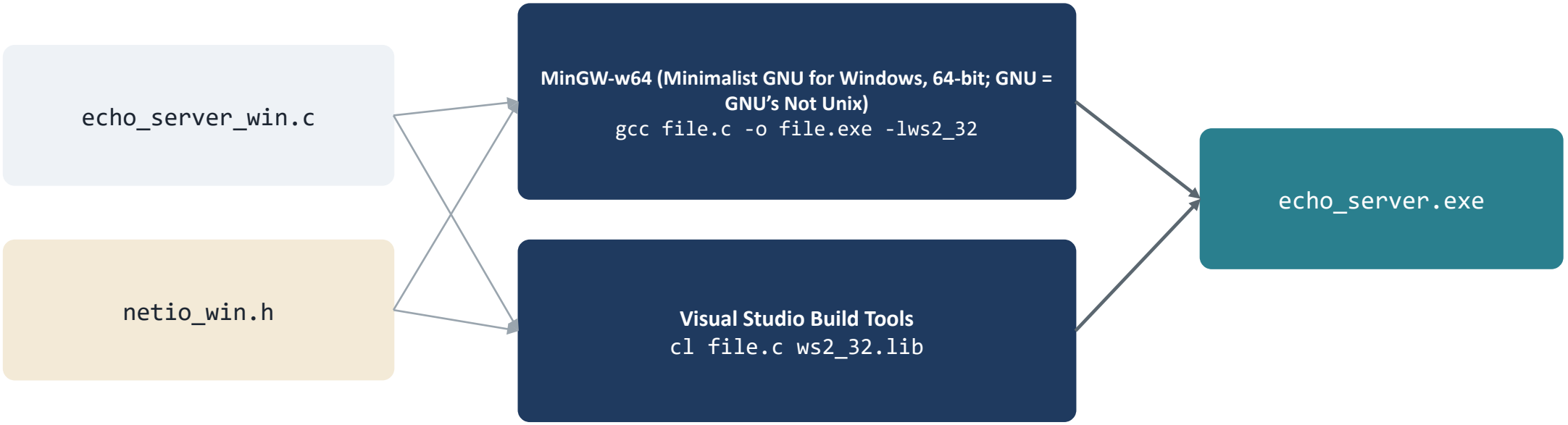
Sarcinile 2–3 · Client și server

Sarcina 5 · Un flux nu e un șir de mesaje

Sarcina 8 · Erori și SO_EXCLUSIVEADDRUSE

SO_EXCLUSIVEADDRUSE (opțiune de socket: folosire exclusivă a adresei) · POSIX (Portable Operating System Interface)

Instrumente · Compilarea unui program Winsock



legare la 127.0.0.1 (doar această mașină)
fără solicitare din partea Windows Firewall — varianta folosită în laborator

legare la toate adresele (INADDR_ANY: orice adresă locală)
Windows cere permisiunea firewall-ului la prima rulare

Ideea-cheie Fără -lws2_32 apar erori de legare „undefined reference”; legarea la 127.0.0.1 evită întrebarea firewall-ului.

Winsock vs POSIX: ce se schimbă

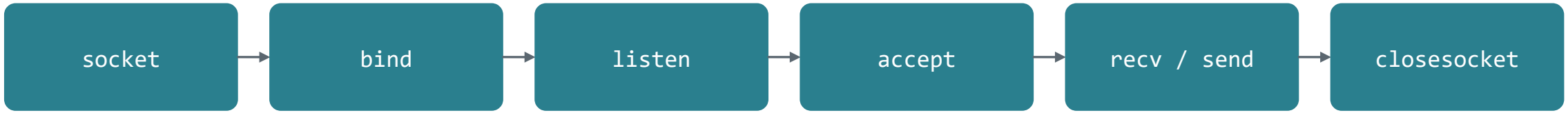
Linux	Windows
(nimic)	WSAStartup() … WSACleanup()
int fd	SOCKET s
-1 la eșec	INVALID_SOCKET / SOCKET_ERROR
close(fd)	closesocket(s)
errno, perror()	WSAGetLastError()
read/write sau recv/send	doar recv/send, lungimi de tip int
SIGPIPE (semnal: conductă întreruptă — broken pipe)	niciun semnal; send eșuează

Ideea-cheie Orice program Winsock începe cu WSAStartup, închide socket-urile cu closesocket și citește erorile cu WSAGetLastError.

WSAGetLastError (ultima eroare Windows Sockets) · WSAStartup (pornește Windows Sockets) · WSACleanup (oprește Windows Sockets) · POSIX (Portable Operating System Interface) · fd (file descriptor, descriptor de fișier)

Viața unui program Winsock

WSAStartup(MAKEWORD(2, 2)) — încarcă Winsock versiunea 2.2



după fiecare apel: comparați cu **INVALID_SOCKET** sau **SOCKET_ERROR**; la eșec afișați **WSAGetLastError()**

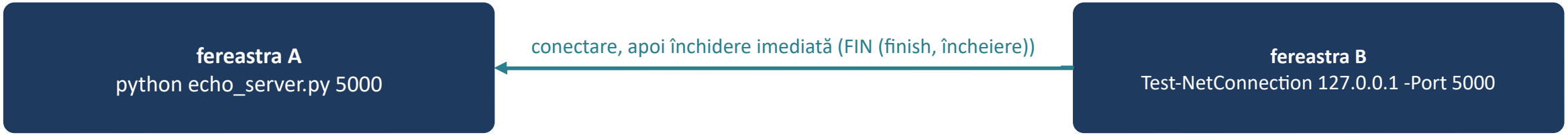
de exemplu 10061 = conexiune refuzată · 10048 = adresă deja folosită · 10054 = conexiune resetată

WSACleanup() — eliberează Winsock

Ideea-cheie Fără WSAStartup, fiecare apel eșuează cu eroarea 10093 (neinițializat).

WSAGetLastError (ultima eroare Windows Sockets) · WSAStartup (pornește Windows Sockets) · WSACleanup (oprește Windows Sockets)

Sarcina 1 · Un socket de ascultare văzut din PowerShell



Get-NetTCPConnection înainte

```
Listen  127.0.0.1:5000
```

în timpul testului

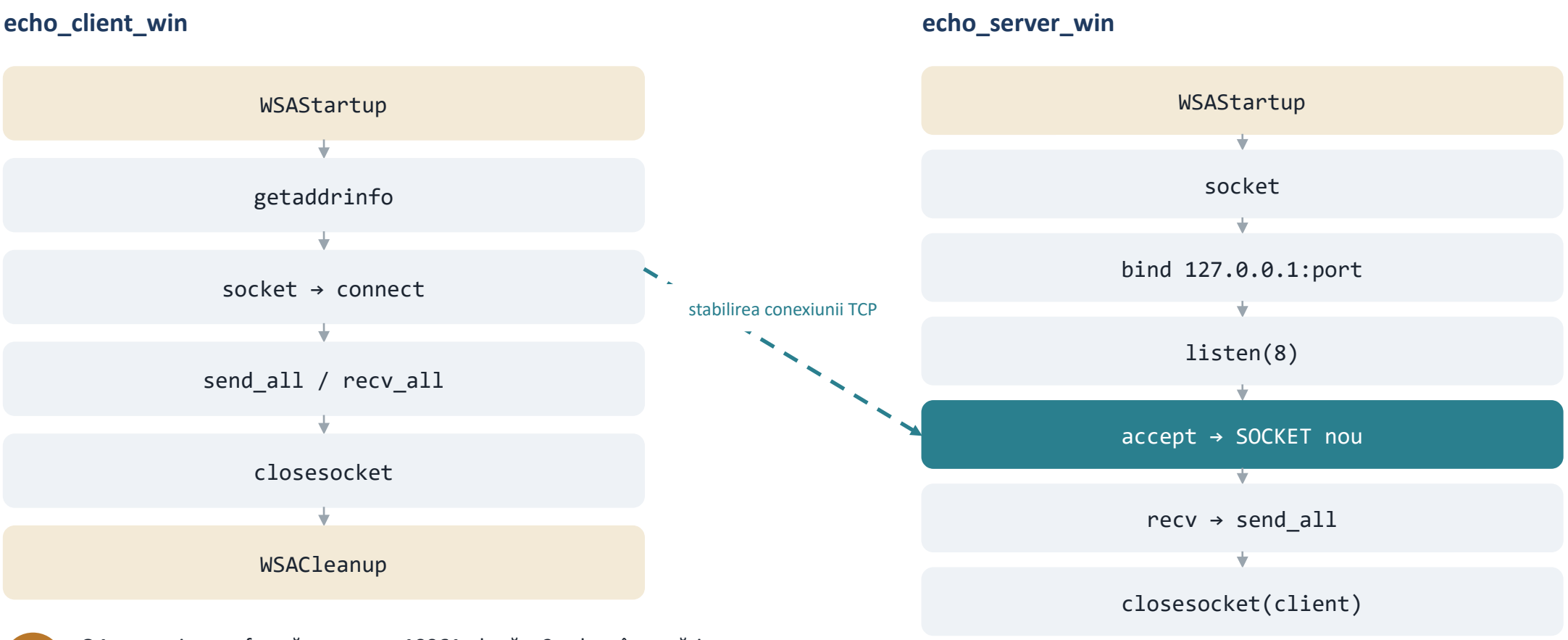
```
Established  127.0.0.1:5000 ⇔ 127.0.0.1:5xxxx
```

1 Q1–Q2 · portul clientului este ales de Windows; serverul afișează „client closed after 0 bytes” când sosește FIN.

Ideea-cheie Get-NetTCPConnection este echivalentul Windows al lui ss: stări, cvadruple și procesul proprietar.

ss (socket statistics)

Sarcinile 2–3 · Clientul și serverul Winsock, alăturate

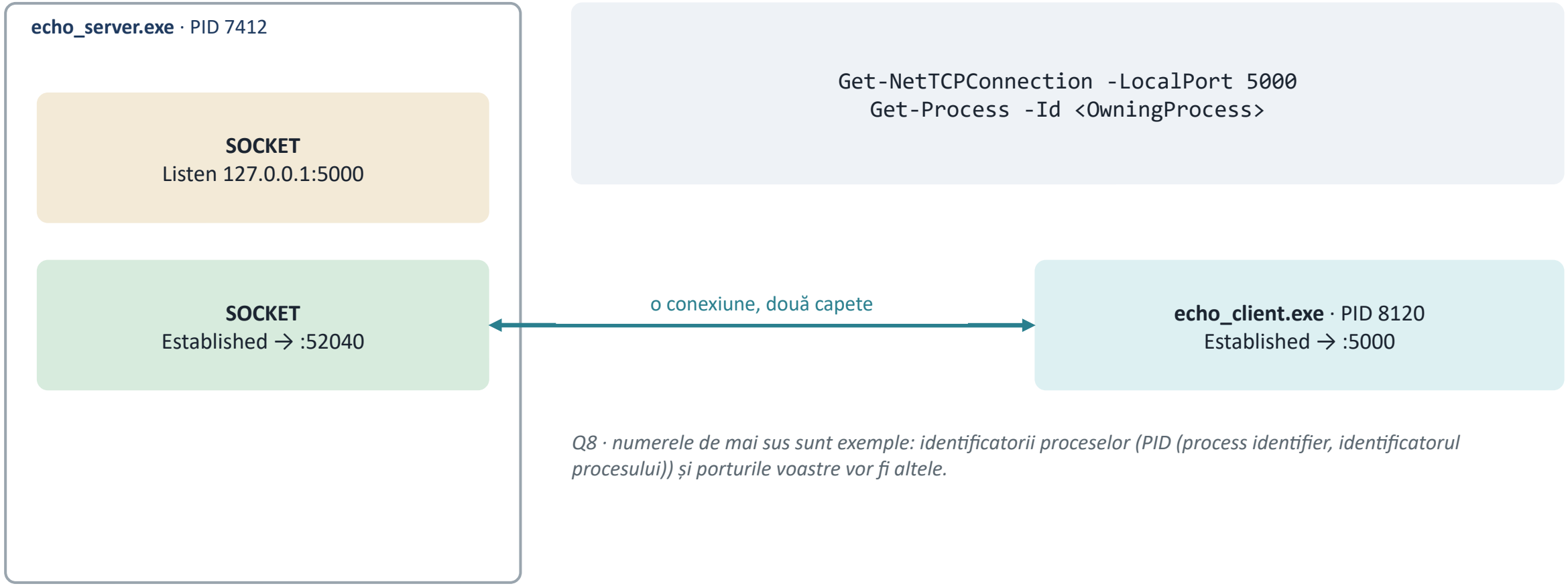


1 Q4 · conexiune refuzată = eroarea **10061**, după ≈ 2 s de reîncercări.

Ideea-cheie Q5–Q7 · Aceiași pași ca pe Linux; al doilea client așteaptă în coadă; fără htons portul este greșit.

WSAStartup (pornește Windows Sockets) · WSACleanup (oprește Windows Sockets) · htons (host to network short, gazdă → rețea, 16 biți) · TCP (Transmission Control Protocol)

Sarcina 4 · Socket-urile văzute de Windows

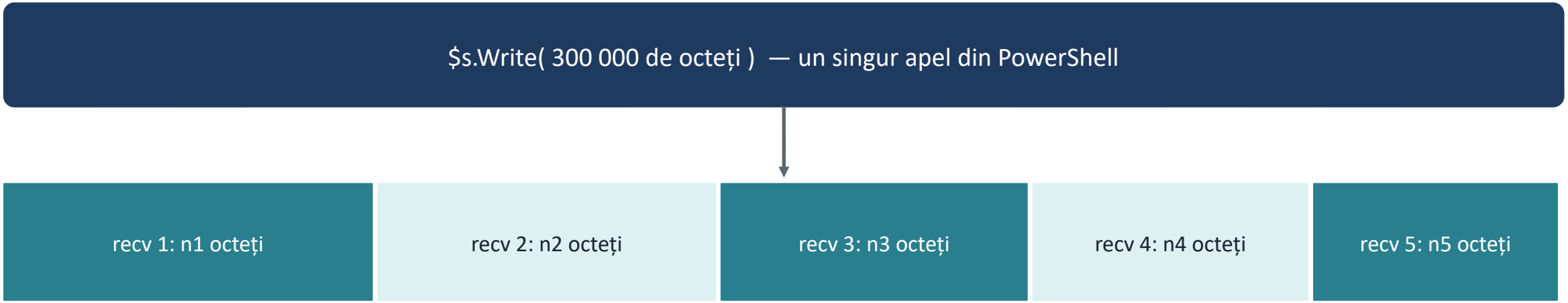



```
Get-NetTCPConnection -LocalPort 5000
  Get-Process -Id <OwningProcess>
```

Q8 · numerele de mai sus sunt exemple: identificatorii proceselor (PID (process identifier, identificatorul procesului)) și porturile voastre vor fi altele.

Ideea-cheie Fiecare capăt al unei conexiuni este un socket deținut de un proces; PowerShell le asociază prin OwningProcess.

Sarcina 5 · O scriere din PowerShell, mai multe recv-uri



ilustrație: măsurați singuri dimensiunile reale — ele se schimbă de la o rulare la alta

1. Q9 · n1 + n2 + … = 300 000, dar niciun recv nu trebuie să fie egal cu scrierea.

2. Q10 · cu o memorie tampon de 7 octeți, serverul trimite ecoul în bucăți de 7 octeți; clientul C le reasamblează cu recv_all.

Ideea-cheie Pe Windows exact ca pe Linux: TCP livrează un flux, nu scrierile voastre.

TCP (Transmission Control Protocol)

Sarcinile 6–7 · Portarea serverelor cu încadrare pe Winsock

neschimbat

memorie tampon + memchr pentru linii

send_msg / recv_msg pentru prefixe de lungime

htonl / ntohl (host ↔ network long, gazdă ↔ rețea, 32 de biți)

se schimbă

WSAStartup / WSACleanup

int → SOCKET

close → closesocket

errno → WSAGetLastError()

legare cu ws2_32

1 Q11–Q14 · aceleași răspunsuri ca pe Linux: ultima linie incompletă nu primește răspuns; prefixul de lungime găsește fiecare graniță.

Ideea-cheie Încadrarea este logică de protocol, independentă de sistemul de operare; diferă doar scrierea interfeței de programare.

WSAGetLastError (ultima eroare Windows Sockets) · WSAStartup (pornește Windows Sockets) · WSACleanup (oprește Windows Sockets) · htonl (host to network long, gazdă → rețea, 32 de biți) · ntohl (network to host long, rețea → gazdă, 32 de biți)

Sarcina 8 · Erorile pe Windows, puse în corespondență cu Linux

eveniment	Windows	Linux
conexiune refuzată	10061 WSAECONNREFUSED, după ≈ 2 s	ECONNREFUSED (eroare: conexiune refuzată), imediat
adresă deja folosită	10048 WSAEADDRINUSE	EADDRINUSE (eroare: adresă deja folosită)
partenerul a fost oprit forțat	10054 WSAECONNRESET	ECONNRESET (eroare: conexiune resetată)
scriere după închidere	10053 / 10054, fără semnal	SIGPIPE (semnal: conductă întreruptă — broken pipe) sau EPIPE (eroare: conductă întreruptă — broken pipe)

Ideea-cheie Q15 · Căutați numărul, apoi numiți evenimentul TCP: RST, FIN sau un port deja ocupat.

WSAECONNREFUSED (eroare Windows Sockets: conexiune refuzată) · WSAEADDRINUSE (eroare Windows Sockets: adresă deja folosită) · WSAECONNRESET (eroare Windows Sockets: conexiune resetată) · TCP (Transmission Control Protocol) · FIN (finish, încheiere) · RST (reset, resetare)

Sarcina 8 · Q16 · SO_REUSEADDR înseamnă altceva pe Windows

cu SO_REUSEADDR pe Windows

serverul A
portul 5000

programul B
tot portul 5000!

Amândouă se leagă la același port.

Nu este definit care îl primește pe următorul client — B poate **fura** conexiunile lui A.

cu SO_EXCLUSIVEADDRUSE

serverul A
portul 5000

programul B
bind → 10048

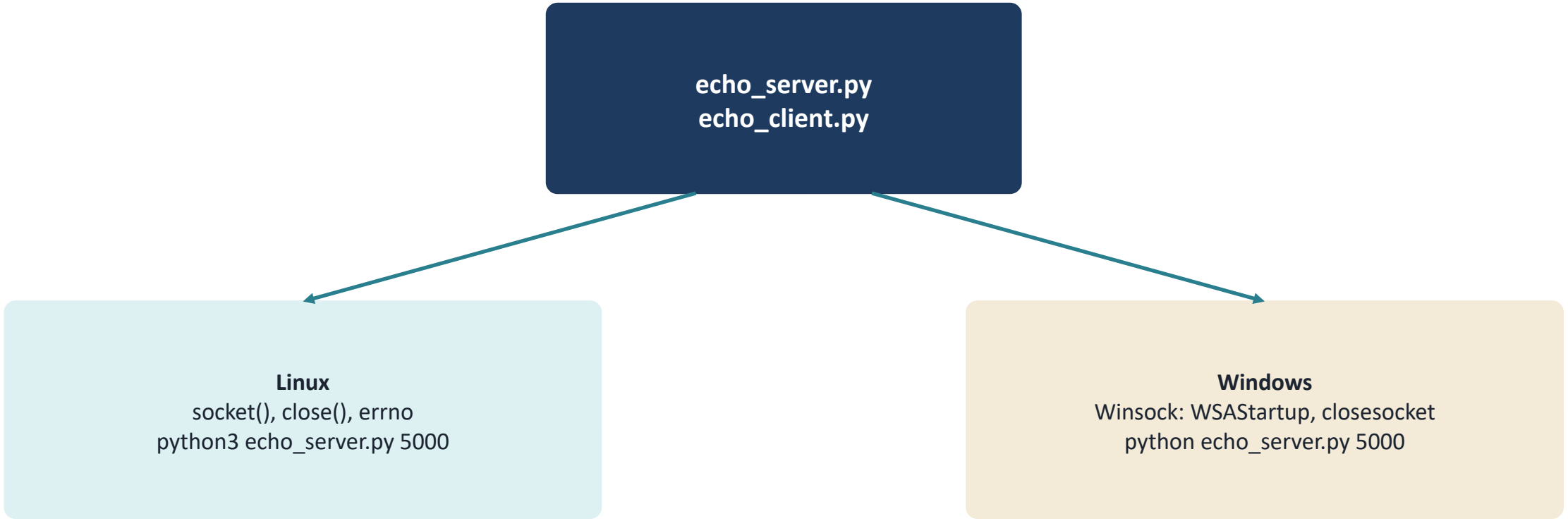
Portul îi aparține doar lui A.

De obicei Windows permite unui server **repornit** să se lege din nou imediat, deci motivul de pe Linux pentru SO_REUSEADDR nu se aplică.

Ideea-cheie Același nume de opțiune, risc opus: pe Windows protejați porturile serverelor cu SO_EXCLUSIVEADDRUSE.

SO_EXCLUSIVEADDRUSE (opțiune de socket: folosire exclusivă a adresei) · SO_REUSEADDR (opțiune de socket: refolosirea adresei)

Un singur fișier Python, două sisteme de operare



1 recv_all rămâne necesar: un recv poate întoarce mai puțini octeți decât s-au trimis, pe ambele sisteme.

Ideea-cheie Conceptele de socket sunt portabile; doar scrierea în C a interfeței de programare nu este.

WSAStartup (pornește Windows Sockets)

Săptămâna viitoare

Cursul 4

cookie-uri și stare · cache web și GET condiționat · HTTP/2 și HTTP/3 · rețele de distribuire a conținutului

Seminarul 4

probleme de cache și de temporizare

Laboratorul 4

socket-uri UDP (User Datagram Protocol) · ordinea octeților în practică

Ideea-cheie Abilitățile de încadrare și de ordine a octeților din această săptămână sunt exact ce le trebuie programelor UDP de săptămâna viitoare.

HTTP (HyperText Transfer Protocol)